From Abstracts to Rooms: Poster Session Assignment by Semantic Similarity and Optimisation



Enes Sejdini
BSc (Hons)

A dissertation submitted for the degree of
Master of Science in Data Science

Supervised by *Dr. Pascal Welke*

School of Computing and Communications
Lancaster University Leipzig

August, 2026

Declaration

I declare that the work presented in this dissertation is, to the best of my knowledge and belief, original and my own work. The material has not been submitted, either in whole or in part, for a degree at this, or any other university. Estimated word count is: **19287**

Name: **Enes Sejdini**
Date: **August, 2026**

From Abstracts to Rooms: Poster Session Assignment by Semantic Similarity and Optimisation

Enes Sejdini, BSc (Hons).
School of Computing and Communications, Lancaster University
A dissertation submitted for the degree of *Master of Science* in Data Science.
August, 2026

Abstract

Large conference poster sessions may contain thousands of papers and rooms with different capacities, which makes it difficult to place related work together manually. This dissertation investigates whether a program can take any conference's papers and room sizes and return a feasible seating plan in which each room contains papers on similar topics. The proposed pipeline uses paper abstracts as its main input. It represents them with semantic embeddings, groups them using Ward's hierarchical clustering, and assigns whole clusters to rooms through capacity-constrained optimisation. Three embedding models and five heuristic assignment methods were compared, leading to the selection of all-MiniLM-L6-v2 and GRASP for the final tool. The system was developed on 5,286 NeurIPS 2025 papers and then applied with the same main stages to a separate sample of 10,000 arXiv papers from nine broad fields. The final NeurIPS layout achieved a pair-weighted within-room cosine similarity of 0.3395, compared with a matching random-assignment mean of 0.3040. The arXiv layout achieved 0.1735, compared with 0.0658. In both experiments, every retained paper was assigned, all room capacities were respected, and no room was left empty. These results show that the pipeline produced feasible layouts with higher within-room similarity than matched random assignments for the two evaluated datasets. Overall, the results provide clear evidence that the pipeline can produce feasible layouts with higher semantic similarity than random assignment across datasets of different sizes and topic distributions. Future evaluation on additional conferences, supported by feedback from visitors, could strengthen these findings and guide further improvements in how spare room capacity is distributed.

Acknowledgements

I would like to express my sincere gratitude and appreciation to my supervisor, Dr. Pascal Welke. Without his unwavering guidance and support throughout, this research project would not have been completed. Thank you for the time you gave me, for the honest feedback on every draft, and for trusting me with my own choices while making sure I never went too far in the wrong direction. Above all, thank you for your expertise and mentorship throughout my academic journey. Your way of stripping a problem down to something simple and solid is the biggest thing I take away from all of this.

I am also grateful to my colleagues and classmates, who have provided me with valuable feedback and support throughout all this process.

Last but not least, I would like to thank my parents, my fiancée, and my brother. To my parents, thank you for raising me, for allowing me every chance you never had, and for making me who I am today. To my fiancée, thank you for bringing out the best in me and for pushing me to become the best version of myself, even at times when it felt impossible. To my brother, thank you for always being beside me, and for making life more enjoyable and more beautiful along the way.

I am honored to be part of this journey, and I look forward to this study being of help and use to everyone that comes across it.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Problem statement and research question . . . 1
- 1.3 Relevance . . . 2
- 1.4 Proposed approach . . . 2
- 1.5 Contributions . . . 3
- 1.6 Results and interpretation . . . 3

2 Literature Review **4**
- 2.1 Conference Session and Poster Assignment . . . 4
- 2.2 Representing Scientific Text as Vectors . . . 5
- 2.3 Hierarchical Clustering of Documents . . . 6
- 2.4 Capacitated Grouping and Clique Partitioning . . . 6
- 2.5 Metaheuristics for Partitioning Problems . . . 7
- 2.6 Benchmarking Algorithms . . . 7
- 2.7 Summary and Positioning . . . 7

3 Pipeline Overview **9**
- 3.1 Architecture . . . 9
- 3.2 Why This Decomposition . . . 10
- 3.3 Notation and Terminology . . . 11
- 3.4 Evaluation Strategy . . . 11

4 Data Collection **12**
- 4.1 The NeurIPS 2025 Dataset . . . 12
- 4.2 The Paraphrase Evaluation Set . . . 13
- 4.3 The Demonstration Dataset . . . 13

5 Embedding Model Selection **15**
- 5.1 Why These Three . . . 16
- 5.2 The Test . . . 17

5.3 Hardening the Test . . . 18
5.4 Results . . . 19
5.5 The Decision . . . 19

6 Clustering **21**
6.1 The Method: Ward's Hierarchical Clustering . . . 22
6.2 How Many Groups . . . 23
6.3 The Result . . . 24

7 Room Assignment **26**
7.1 The Problem, Written Out . . . 26
7.2 The Exact Attempt . . . 30
7.3 The Turn to Heuristics . . . 31
7.4 The Five Methods . . . 32
7.4.1 GRASP . . . 32
7.4.2 Tabu Search . . . 32
7.4.3 Matheuristic . . . 32
7.4.4 Variable Neighborhood Search . . . 33
7.4.5 Memetic Algorithm . . . 34
7.5 The Choice: GRASP . . . 34

8 The Layout Tool **36**
8.1 Input . . . 38
8.2 Hierarchical Clustering . . . 40
8.3 Heuristics . . . 42
8.4 What the Tool Gives Back . . . 43
8.5 The Demonstration . . . 44

9 Results, Discussion and Analysis **47**
9.1 The Final Result . . . 47
9.2 Similarity Analysis for Each Room . . . 48
9.3 How the Heuristics Handled Exact Fill . . . 49
9.4 How Well the Method Choice Held Up . . . 50
9.5 Did the Tool Produce a Good Layout for Conferences? . . . 51
9.6 Expectations vs Actual Outcomes . . . 52

10 Conclusion **54**
10.1 Limitations . . . 55
10.2 Future Work . . . 55

References **57**

List of Figures

3.1 The four stages of the pipeline. The input is any conference's papers as a csv file with one abstract per paper. NeurIPS 2025, retrieved from OpenReview, is the dataset used throughout this dissertation. The dashed arrows show the two non-linear parts: the evaluation of the finished layout against a random baseline, and the loop that re-cuts the cluster tree when a cluster does not fit the largest room. 10

4.1 Accepted NeurIPS 2025 papers per primary area, from 1,685 papers down to 54. 13
4.2 The 10,000 sampled arXiv papers per group, from 1,182 down to 1,001. . . . 14

List of Tables

4.1 The two datasets side by side. 12
4.2 Construction of the demonstration dataset. 14

5.1 The three candidate models, each used exactly as delivered. 16
5.2 Results over the 250 paraphrases. Lower rank is better. 19

6.1 What the clustering costs, for all 5,286 papers. 22
6.2 Common alternatives and the required property each one lacks. 23
6.3 The 529 groups in numbers. 24
6.4 Example groups: the three largest, the tightest, and the loosest. 24

7.1 GRASP over 20 attempts of 60 seconds. 32
7.2 Tabu Search over 20 attempts of 60 seconds. 33
7.3 Matheuristic over 20 attempts of 60 seconds. 33
7.4 VNS over 20 attempts of 60 seconds. 33
7.5 Memetic algorithm over 20 attempts of 60 seconds. 34
7.6 All five methods over the same 20 attempts of 60 seconds. Gaps are to the best layout found, not to a proven optimum. 34
7.7 Pairwise Wilcoxon tests, Holm-corrected. Values below 0.05 mean a real difference. 35

8.1 One run of the tool from start to finish, and where each step was decided. . . 37
8.2 What is fixed in a run of the tool, and what is not. 37
8.3 The five files of the tool and what each one does. 38
8.4 The columns the tool looks for in the papers file. 39
8.5 The rules the tool works out by itself, and what they come to on each dataset. 41
8.6 The two runs of the tool. 44
8.7 room_summary.csv as produced for the NeurIPS run. 45
8.8 room_summary.csv as produced for the demonstration run. 46

9.1 The five promises of Section 3.1 checked against the delivered plans. 48

Chapter 1

Introduction

This dissertation studies how paper abstracts can be used to organise a large conference poster session. The aim is to place related papers in the same rooms while respecting each room's capacity. The proposed pipeline combines text embeddings, hierarchical clustering, and optimisation, then measures the completed layout. This chapter introduces the problem, research question, approach, contributions, and main results. Chapters 2–7 develop the method, Chapters 8 and 9 present and evaluate the completed layouts, and Chapter 10 gives the conclusions, limitations, and future work.

1.1 Motivation

A large poster session may contain thousands of papers. Visitors can find relevant work more easily when papers about similar topics are placed close together. Creating this layout manually becomes difficult when the venue has many rooms with different capacities.

The NeurIPS 2025 dataset used in this dissertation contains 5,286 accepted papers for twelve rooms. Subject labels cannot solve the problem alone. The largest subject area contains 1,685 papers, while the largest room holds 700, so large areas must be divided and smaller areas combined. CVPR already uses paper embeddings to group similar posters (CVPR, 2024), but its public description does not provide a numerical evaluation of the final layout. This dissertation therefore studies both the assignment and its measured quality.

1.2 Problem statement and research question

The layout has three hard rules. Every retained paper must be assigned to exactly one room, no room may exceed its capacity, and no room may be empty. Within these rules, the aim is to place similar papers together. The first three conditions can be checked directly, while semantic similarity must be represented numerically and optimised across a very large set of

possible layouts.

The problem is related to clique partitioning, where items are divided into groups to maximise the similarity within them. Clique partitioning is NP-hard (Grötschel and Wakabayashi, 1990). The poster-session problem adds room capacities and a minimum occupancy rule. This leads to the main research question:

> *Can a program take any conference's papers and any set of room sizes and return a seating plan in which every room holds papers about similar things?*

The question is tested on two datasets and two capacity settings. The tool is designed for different conferences, but its performance was measured only on the two datasets and room configurations used in this study.

1.3 Relevance

Chapter 2 reviews five areas relevant to this study: conference scheduling, text embeddings, document clustering, capacitated grouping, and heuristic benchmarking. Previous research has used optimisation for conference sessions and text-based methods for identifying paper topics. Burke and Sabatta (2015) used topic modelling for session assignment, while Bulhões, Correia, and Subramanian (2022) developed a clustering-based scheduling model. The literature also provides the embedding, clustering, partitioning, and evaluation methods used in this project. These studies show that the individual building blocks are established. This dissertation examines how they can be integrated into one evaluated pipeline for capacity-limited poster rooms.

1.4 Proposed approach

The pipeline has four stages. First, the paper data are prepared and the abstract is kept as the main text. Second, each abstract is converted into an embedding, which is a numerical vector representing its meaning. MiniLM was selected after a retrieval comparison with SPECTER2 and Aspire. Third, Ward hierarchical clustering turns thousands of papers into smaller topic groups. Fourth, the groups are assigned to rooms while keeping every capacity rule and each group together.

An exact CP-SAT formulation was tested before five heuristic methods were compared. GRASP was selected for the final tool based on its cluster-level objective, stability, and simpler implementation. The optimiser compares cluster centroids, while the final paper-level score compares the actual paper embeddings. These related measures are not identical. The final score is also compared with five seeded random assignments that keep the same room occupancies.

1.5 Contributions

This dissertation makes five contributions. First, it provides a complete paper-to-room pipeline. Second, it compares three embedding models. Third, it benchmarks five room-assignment heuristics. Fourth, it evaluates the final NeurIPS and arXiv layouts against matching random-assignment baselines. Fifth, it applies the same main pipeline stages to a second dataset containing 10,000 arXiv papers. Together, these contributions cover the complete process from representing paper abstracts to evaluating the final room layouts.

1.6 Results and interpretation

The final NeurIPS layout achieved a within-room similarity of 0.3395, compared with a random-assignment score of 0.3040. This was an absolute increase of 0.0355, or approximately 12 per cent. The arXiv layout achieved 0.1735, compared with a random score of 0.0658. This was an absolute increase of 0.1077, or approximately 164 per cent. Both layouts assigned every paper included in the experiments, respected all room capacities, and left no room empty.

The arXiv layout also showed that feasibility does not always produce a balanced use of the available space. Room 9 received only four papers despite having a capacity of 750.

In the model comparison, all-MiniLM-L6-v2 achieved the best average retrieval rank and was more than 150 times faster than both tested scientific models. In the room-assignment comparison, GRASP achieved the highest average objective and the lowest spread, which supported its selection for the final tool. The exact-solver attempt ran for four hours without producing a proven result, which led to the use of time-bounded heuristic methods.

Overall, the results provide a positive answer to the research question for both evaluated datasets. Each layout achieved a higher similarity than its own random-assignment baseline while respecting the stated room rules. The arXiv experiment further shows that the same main pipeline can process a larger dataset with a different topic distribution, distinguish between broad scientific fields, and identify similarities among papers within related subjects. To ensure higher confidence in final results, this study is looking to expand in experimenting direct experience of visitors in real conferences.

Chapter 2

Literature Review

This study builds on a few areas of research. This chapter goes through them one at a time. It starts with conference scheduling. Then how text is turned into numbers. Then how papers are grouped. Then the hard problem underneath. And last, how the methods that solve it are compared in a fair way. The final section puts the pieces together and shows the gap this study fills.

2.1 Conference Session and Poster Assignment

The problem of organizing a scientific conference by computational means is not new. In an early study, Eglese and Rand (1987) surveyed the attendees of a 1985 conference to express which seminars they wished to attend, and then built a schedule that would allow the fullfillment of as many of these wants as possible. From that point on, the organizing of poster placements in a conference has become received a great deal of attention leading to the establishment of a large amount of operational research literature, where this problem has mainly been dealt as an integer linear problem. Research such as Stidsen, Pisinger, and Vigo (2018) describe a tool used to schedule the EURO-k conferences, events with more than two thousand presentations, by using optimization models to place related research buildings close to each other. Riquelme et al. (2022) focused on a "track-based" version of the problem—sorting choices into specific paths. Instead of finding an exact optimal solution, which is always computationally slow, they used a "simulated annealing heuristic," which is a smart shortcut algorithm. This shortcut found comparable solutions in a fraction of the time. Closest to our setting, Bulhões, Correia, and Subramanian (2022) define a general session scheduling problem whose objective is to maximize the similarity of papers put next to each other. They prove that this problem is NP-hard, and solve real conference instances with branch-and-cut and branch-cut-and-price methods. While this exact approach guarantees an optimal solution, it is computationally expensive, leaving room to develop faster methods that produce comparable results.

There is also a smaller line of research that finds topics by reading the actual paper texts, not by relying on pre-set labels or user preferences. Burke and Sabatta (2015) use Latent Dirichlet Allocation (LDA) directly to the submitted documents to learn the conference topics. They then schedule the parallel tracks so that simultaneous sessions cover the most different topics possible. By maximizing the distance between session topics, they ensure attendees are not forced to choose between two similar talks. In current practice, large venues face the same task at much greater scale: For example, CVPR (CVPR, 2024) groups its accepted posters by similarity within each session. To decide which posters are similar, they use SPECTER embeddings and then reduce the size of those vectors down to a single line with one-dimensional t-SNE. Posters that land close together on that line are put in the same group . This is, to our knowledge, the closest real-world precedent for the problem we study; however, it is a projection heuristic rather than an explicit optimization, and the quality of the resulting layout is not measured.

Put together, the literature has largely pursued conflict avoidance and preference satisfaction. The topical coherence of a capacity-limited physical room, treated as an explicit optimization objective in itself, remains largely unexplored.

2.2 Representing Scientific Text as Vectors

Sentence embedding models map a text to a fixed-length vector such that semantically similar texts receive nearby vectors. Sentence-BERT introduced the now-standard approach of fine-tuning a pretrained transformer with siamese network structures, producing sentence vectors that can be compared directly with cosine similarity (Reimers and Gurevych, 2019). The model used later in this work, all-MiniLM-L6-v2, combines this training scheme with the MiniLM distillation method, which compresses a large teacher transformer into a six-layer student with 384-dimensional outputs (Wang et al., 2020). In contrast, a family of models exists specifically for scientific documents: SPECTER fine-tunes SciBERT on the citation graph, treating papers that cite one another as similar (Cohan et al., 2020); its successor SPECTER2 is trained on over six million citation triplets and attaches task-specific adapters (Singh et al., 2023); and Aspire models fine-grained similarity by matching individual sentences between papers (Mysore, Cohan, and Hope, 2022). The landscape is wider than the three models examined later: contrastive objectives such as SimCSE learn general-purpose sentence embeddings without any labelled pairs (Gao, Yao, and Chen, 2021), and SciNCL replaces SPECTER's discrete citation relations with sampling over citation-graph embeddings (Ostendorff et al., 2022). These models were trained on different signals and therefore encode different notions of what makes two papers similar, a distinction whose practical consequences we examine in Chapter 5.

2.3 Hierarchical Clustering of Documents

Clustering methods vary not solely in quality but in their structural properties, and these properties determine how well a method fits a given task. Hierarchical agglomerative clustering starts from every point as its own group and repeatedly merges the closest pair, producing a tree that can afterwards be cut at any level to yield any number of clusters; Ward's method chooses each merge so that the increase of the within-group variance is minimal, which favours compact groups (Ward, 1963). The k-means algorithm instead partitions the data into a number of groups that must be fixed in advance, and because it starts from a random initialization, its result can vary from run to run, which reduces reproducibility (MacQueen, 1967; Jain, 2010). Density-based methods such as HDBSCAN extract clusters of varying density from a hierarchy of density estimates (Campello, Moulavi, and Sander, 2013; McInnes, Healy, and Astels, 2017); a defining feature of this family is that points in low-density regions are labelled as noise and left outside every cluster. Finally, probabilistic topic models such as Latent Dirichlet Allocation(LDA) describe each document as a soft mixture of topics rather than assigning it to exactly one group (Blei, Ng, and Jordan, 2003).

In this work, clustering is judged by practical fit rather than benchmark accuracy: every poster must be placed (no unassigned outliers), the number of groups must be freely re-cut to match physical room capacities, and the outcome must be deterministic so that our layout heuristics remain stable. Chapter 6 derives the choice of hierarchical clustering directly from these three requirements.

2.4 Capacitated Grouping and Clique Partitioning

At its combinatorial core, our task is the clique partitioning problem: assign items to mutually exclusive groups so that the total similarity within groups is maximized. (Grötschel and Wakabayashi, 1989) introduced this formulation as a clustering model and proposed cutting plane methods for its solution. The problem was later shown to be NP-hard (Grötschel and Wakabayashi, 1990), and the geometric structure of its feasible solutions has been studied extensively (Oosten, Rutten, and Spieksma, 2001). Even with modern commercial solvers, an exact solution becomes impractical as instances grow, something that (Du et al., 2022) has also concluded through a direct comparison of models and solvers on clique partitioning benchmarks. Our problem in Chapter 7 adds two further restrictions to this core, namely room capacities and a minimum occupancy per room, and regardless of the added rules, it still doesn't remove its NP-hardness.

2.5 Metaheuristics for Partitioning Problems

For partitioning problems, a widely accepted set of metaheuristics exists. GRASP repeatedly builds a solution with a randomized greedy construction and then improves it with local search, keeping the best result over many restarts (Feo and Resende, 1995). Tabu search follows a single trajectory and uses a short-term memory of recently made moves to escape local optima (Glover and Laguna, 1997). Variable neighbourhood search alternates between descending to a local optimum and perturbing the solution with systematically growing shakes (Mladenović and Hansen, 1997). Memetic algorithms evolve a population of solutions through recombination and mutation, with local search applied to the offspring (Moscato and Cotta, 2003). These methods have been applied to clique partitioning itself, for example through iterated tabu search (Palubeckis, Ostreika, and Tomkevičius, 2014), and hybrid approaches additionally exist that embed an exact solver inside a heuristic loop, a design we will go back to in Chapter 7. The metaheuristic family is of course wider than these five; simulated annealing, for instance, has been applied in the conference scheduling setting itself as well(Riquelme et al., 2022).

2.6 Benchmarking Algorithms

All of the methods above are randomized, so a single run of each says little about which one is better. The established standard procedure is therefore to compare distributions of results over multiple independent runs, and to test the differences with non-parametric statistics, since the outcomes of heuristics cannot be assumed to follow a normal distribution. Demšar (2006) recommends exactly this design: the Wilcoxon signed-rank test for comparing two methods on paired results, and the Friedman test as an omnibus test when several methods are compared at once. Running many tests together creates a problem of its own: each test has a small chance of a false alarm, so across ten pairwise comparisons at least one "significant" result is quite likely to be fake. The procedure of Holm (1979) fixes this by making each individual test stricter, keeping the whole table at the usual five percent error level, and García and Herrera (2008) show how to apply it when every method is compared against every other, which is our case. Chapter 7 follows exactly this protocol; it is also why every method there receives the same time budget and the same random seeds, since the tests compare methods seed by seed, and that pairing is only fair if both methods faced identical conditions on each seed.

2.7 Summary and Positioning

Together, the literature is divided into three paths.First, the operational research norm schedules conferences well, but its objectives are preventing overlapping sessions and

satisfying preferences, not the topical coherence of a room. Second, a smaller line of research reads the papers themselves to find topics but stops at abstract groupings without rooms or seat capacities. Third, in practice, large venues already assign posters with embedding-based heuristics, without measuring the quality of the result. All building blocks our pipeline needs exists on its own: sentence embeddings, hierarchical clustering, the clique partitioning problem, metaheuristics, and a statistical protocol for comparing them fairly. What no prior work does, is combine them, a reproducible pipeline that takes a conference's raw submissions and returns a topically coherent, physically feasible assignment of papers to capacity-limited rooms. That arrangement is what this dissertation provides.

Chapter 3

Pipeline Overview

The pipeline was built to meet five requirements. First, the layout it produces must be topically coherent: papers found in the same room should talk about similar concepts, since this is the entire purpose of the system itself. Second, the layout must be physically feasible, meaning that no room is allowed to hold more papers than it has seats and no room is allowed to be left empty. Third, the pipeline must maintain peak performance at the scale of a real conference, which today means thousands of papers. Fourth, it must be general: the same pipeline, standardized, should accept any conference's papers and any set of room sizes, ensuring reliability at any scale of a scientific conference. Finally, it must be reproducible: every random choice the pipeline makes is controlled by a fixed seed, so repeated runs follow exactly the same search. However, in the heuristic faze only the search is stopped by wall-clock time rather than by a step count, the resulting layout is the same in practice but not guaranteed to be identical, a point Chapter 8 returns to.

3.1 Architecture

The pipeline is made up of four stages, shown in Figure 3.1. First stage obtains the accepted papers of a conference, in our case the abstracts and corresponding metadata of the accepted NeurIPS 2025 submissions. Embedding starts the second stage by converting each abstract into a fixed-length numerical vector, so that the similarity of two papers can be computed as the similarity of their vectors. Clustering groups these vectors into many small topic clusters, by calculating the distance of the vectors with eachother and grouping the closest ones together. Room assignment then packs whole clusters into the available rooms, maximizing the similarity of the clusters that end up together, and an evaluation step measures the quality of the finished layout. Each stage hands its output to the next: stage one delivers the abstracts, stage two turns them into unit-length vectors, stage three groups the vectors into clusters with centroids, and stage four turns the clusters into a paper-to-room layout again based on similarity of papers between clusters.

The four stages follow the standard data science pipeline of collection, representation, modelling and validation. As that framework itself emphasizes, the process is iterative rather than linear: in our pipeline, if clustering produces a cluster larger than the largest room, no feasible layout can exist, and the flow loops back to re-cut the same cluster tree more finely until every cluster fits. The pipeline is demonstrated throughout this dissertation on a single real instance, the 5,286 accepted papers of NeurIPS 2025 assigned to twelve rooms, and the properties of this dataset are described in Chapter 4.

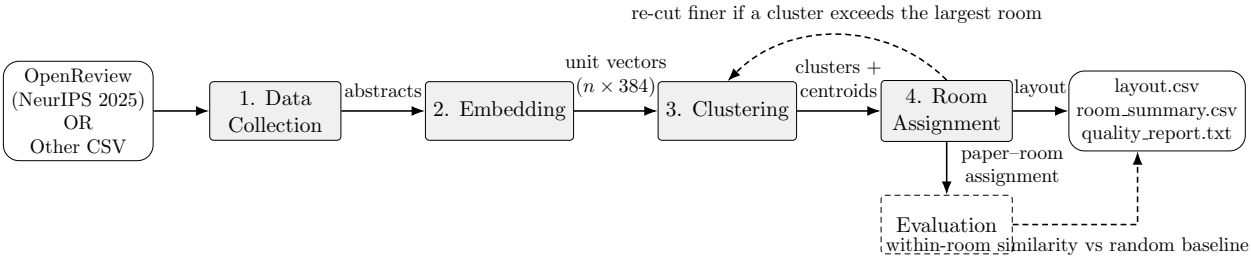


Figure 3.1: The four stages of the pipeline. The input is any conference's papers as a csv file with one abstract per paper. NeurIPS 2025, retrieved from OpenReview, is the dataset used throughout this dissertation. The dashed arrows show the two non-linear parts: the evaluation of the finished layout against a random baseline, and the loop that re-cuts the cluster tree when a cluster does not fit the largest room.

3.2 Why This Decomposition

The central design decision of the pipeline is that papers are not assigned to rooms individually. Instead, they are first grouped into clusters, and the optimization then places the clusters into the given rooms. Three reasons for doing this. The first is scale: assigning 5,286 individual papers to twelve rooms encompasses a search space far greater than what any exact or heuristic method can explore meaningfully in a short time, while a few hundred clusters can simplify the task significantly. The second is signal: a single paper provides little topical information on its own, whereas a cluster of related papers can end up having a stable, recognizable subject, making it easier to optimize. The third is past experiences: nobody walks a poster session with a list of single papers in mind. People think in topics, see the robotics posters, then the medical ones, so the natural focus of a session area the paper subject areas, and the produced clusters end up being building blocks of those areas. Assigning papers one by one could sometimes scatter a topic across several rooms, assigning whole clusters increases the likelyhood of every topic being in the right place.

This decomposition has an honest cost, and we state it here because it is a property of the shape rather than of any individual method: clusters are unchanging. Once formed, a cluster

is decided, so the grouping might interfeer with the flexibility of splitting a topic accross two different rooms. This lack of flexibility is what later makes filling every room to its exact seat count difficult, and its consequences are examined in Chapter 9. Further details on which clustering method forms the clusters, and which optimization method packs them, are also treated in Chapter 6 and Chapter 7 respectively.

3.3 Notation and Terminology

The following notation is used throughout. The conference has n papers and R rooms with seat capacities $c_1, \ldots, c_R$. Clustering produces C clusters with sizes $s_1, \ldots, s_C$, each represented by a centroid, and W denotes the $C \times C$ table of pairwise centroid similarities. A layout is an assignment that maps every cluster to exactly one room. Two terms are kept strictly apart: the *objective* is the cluster-level score that the optimization maximizes, while the *within-room similarity* is the paper-level measure by which the finished layout is assessed. These are all related but not identical measurements, and further details on the importance of each is analysed in Chapter 9. However, the formal optimization model built from this notation is stated at the start of Chapter 7.

3.4 Evaluation Strategy

The pipeline is evaluated using three distinct measures. The first is the optimization objective, which serves as the internal mathematical score that the assignment methods in Chapter 7 compete to maximize. The second is the mean within-room similarity of the actual papers. Because this is calculated independently of the optimizer, it acts as an unbiased reality check of the grouping quality. The third is a baseline control, created by randomly shuffling the exact same papers into the same room sizes. Because all papers at a single conference naturally share a broad topic and vocabulary, any random grouping will inherently yield a baseline level of similarity. Consequently, a raw similarity score is meaningless on its own; it only becomes interpretable when compared directly against this random baseline to prove the algorithm is genuinely finding deeper connections.

Chapter 4

Data Collection

This work uses two datasets with strictly separated roles; Table 4.1 compares them. Every experiment and every design decision, from the model comparison of Chapter 5 to the final layout of Chapter 8, is made on the NeurIPS 2025 corpus. The arXiv sample is used exactly once, in Chapter 8, to run the finished tool on a corpus it was not developed on; nothing touches it before that run. A third, small set is derived from the NeurIPS data: the paraphrase set of Section 4.3.

Table 4.1: The two datasets side by side.

	NeurIPS 2025	**arXiv sample**
Source	OpenReview API	arXiv metadata snapshot
Papers	5,286	10,000
Fields	5	4
Missing values	none	none
Fields the tool reads	Title, Abstract	title, abstract
Reference labels	Primary Area	group, subcategory
Abstract mean	181.6 words	138.4 words
Abstract range	37–516 words	5–344 words
Categories	13 primary areas	9 arXiv groups
Balance	heavily unbalanced	close to balanced
Snapshot	July 2026, frozen	frozen sample, fixed seed
Role	all development	demonstration only

4.1 The NeurIPS 2025 Dataset

The papers come from OpenReview, the platform on which NeurIPS runs its review process. A short Python script, using the official OpenReview client library, requests all submissions

carrying the venue identifier NeurIPS.cc/2025/Conference and writes them to one CSV file, one row per paper. The accepted NeurIPS 2025 submissions were downloaded in July 2026, and the dataset reflects the state of the platform at that time. Only the abstract feeds the pipeline; the title labels the outputs, and the rest is kept as reference only.

Figure 4.1 shows the categories the authors chose for their papers. Deep learning alone holds 1,685 papers, almost a third of the conference, while the three smallest areas hold fewer than sixty each. Because most papers here are already similar, our implementation can show false success thus leading to the need of a second testing dataset.

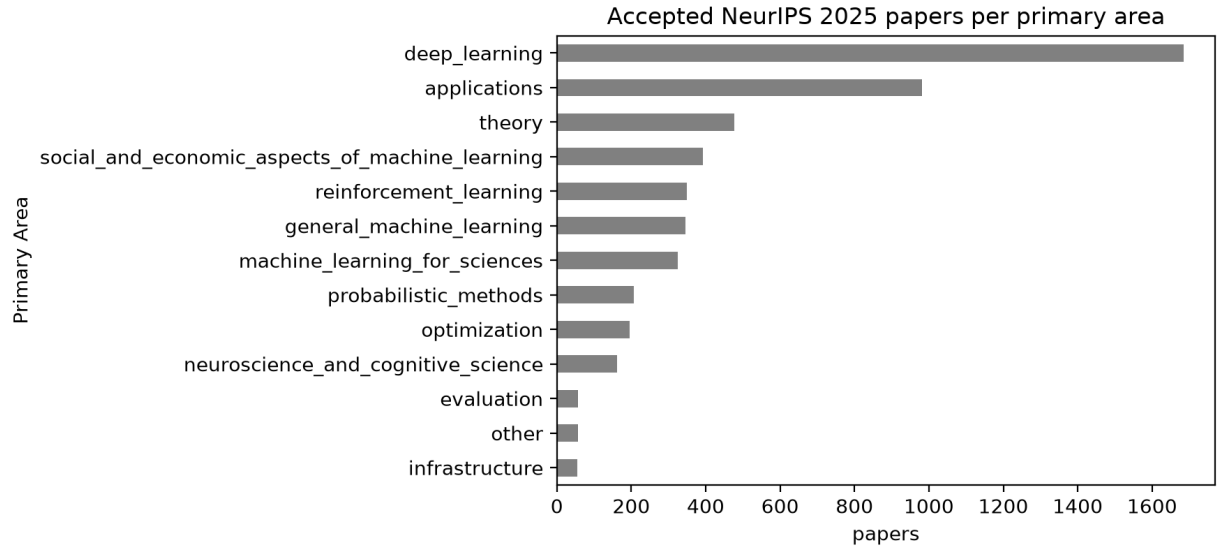


Figure 4.1: Accepted NeurIPS 2025 papers per primary area, from 1,685 papers down to 54.

4.2 The Paraphrase Evaluation Set

Chapter 5 needs a labelled test set for its model comparison. We built one from the NeurIPS data: 250 random abstracts, each rewritten into a single sentence by the Gemma E2B model, run locally through Ollama. A good model should recognise the original paper from that sentence. The words change, the meaning does not. Rerunning the model gives different sentences each time, so the set was written once and kept unchanged.

4.3 The Demonstration Dataset

The NeurIPS corpus is topically narrow, almost a third of it is deep learning. High within-room similarity could come partly from the papers resembling each other, not from the

implementation being good. The finished application is therefore also run once, deliberately, through different dataset, built as Table 4.2 describes. Figure 4.2 shows the outcome: nine fields, close to balanced, the opposite of Figure 4.1.

Table 4.2: Construction of the demonstration dataset.

Source	arXiv metadata snapshot, ∼3 million entries
Selected	the nine top-level arXiv groups
Category used	primary, the first one the authors listed
Excluded	legacy categories from the 1990s
Cap per group	11,000, the size of the smallest group
Within a group	cap split equally among subcategories; undersized ones contribute all they have
Parent set	Over 3 Million papers
Sample	10,000, uniform random, fixed seed
Provided	sampled file ships with the code

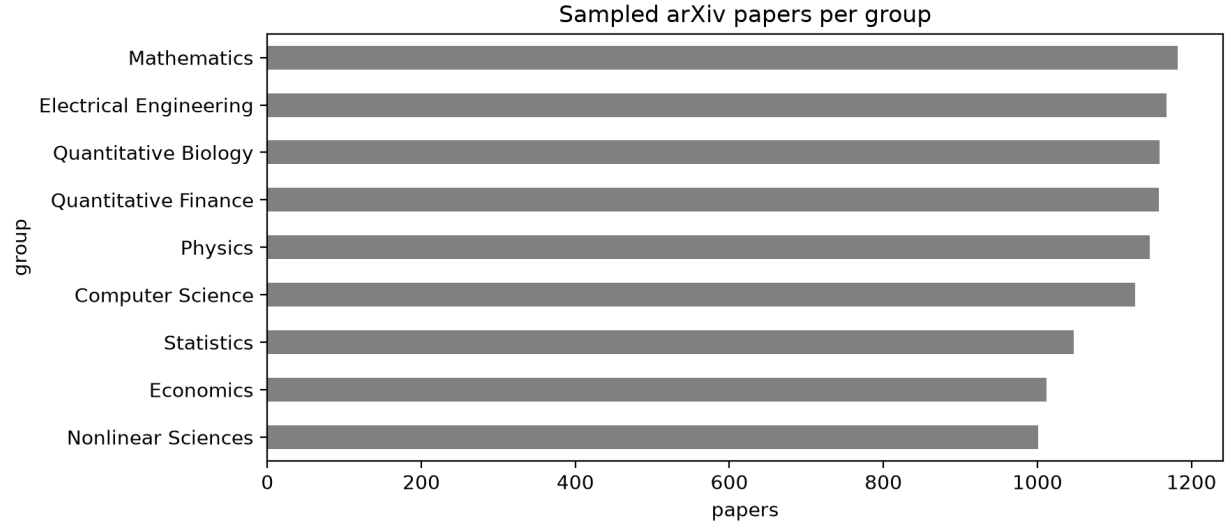


Figure 4.2: The 10,000 sampled arXiv papers per group, from 1,182 down to 1,001.

Chapter 5

Embedding Model Selection

The pipeline turns every abstract into a vector, a task assigned to the final chosen embedding model. This chapter describes how that choice was made. An embedding model reads a sentence and looks at how the words work together to figure out its exact topic. Based on that topic, it outputs a numerical value called a vector. It does this for every piece of text you input, and in the end, the vectors that are closest to each other represent the most similar topics. The abstract is the text given to it, because the abstract is the summary the authors wrote themselves and therefore carries more information about the topic per word than any other part of the paper. Additionally, and because it is short,under two hundred words on average against several thousand for a full paper, it keeps the work small enough for the pipeline to compute the layout in better time and typically without sacrificing quality.

This study compares three candidates, which are listed in Table 5.1. Two of them were built specifically for scientific documents. SPECTER2 learns from citation signals, meaning it treats papers that cite one another as similar (Cohan et al., 2020; Singh et al., 2023), while Aspire learns from the sentences in which two papers are cited together (Mysore, Cohan, and Hope, 2022). The third candidate, all-MiniLM-L6-v2, is a small general-purpose model from the Sentence-BERT family (Reimers and Gurevych, 2019; Wang et al., 2020). It was trained on scientific text as well, but on everyday text alongside it (Sentence-Transformers, 2021), which is expected to help wherever papers describe their work in ordinary rather than technical language, as those in economics and in the social and economic areas of machine learning often do. This setup is built on purpose to test two specialized models against one single model. While the two specialists are each trained on only one kind of language, the single model is trained on both at the same time, allowing us to see which approach works best.

Table 5.1: The three candidate models, each used exactly as delivered.

Model	**Size**	**Dim.**	**Window**	**Metric**
all-MiniLM-L6-v2	22.7M params	384	256 tokens	cosine
SPECTER2	BERT-base	768	512 tokens	Euclidean
aspire-sentence-embedder	BERT-base	768	512 tokens	Euclidean

5.1 Why These Three

A large variety of models can turn text into vectors, but only a filtered amount is right for our task. Three questions do the narrowing: which models can perform this task at all, which types of those models are best, and what the available hardware can run.

The first question removes most models. A language model such as BERT is trained by hiding words in a sentence and having the model guess them. This teaches it a great deal about language, but it never teaches it to arrange texts according to how similar they are, and Reimers and Gurevych (2019) showed the consequence: comparing the output of such a model directly gives poor results, worse even than far simpler methods that ignore word order altogether. A model becomes usable here only after a second stage of training, in which it is shown pairs of related texts and taught to keep them close together while pushing unrelated texts apart. This is why SciBERT, the standard model for scientific text (Beltagy, Lo, and Cohan, 2019), was not considered. Reading science is not the missing part: SciBERT simply never received that second stage, and comparing texts is the only thing it would be needed for.

The second question is which type of models that can do the task do we actually select as best. Among the models that did receive that training, some were trained on scientific papers in particular, and were taught which papers belong together by looking at how researchers cite one another. Models built for text in general are trained on a broad mixture of everyday writing, in which scientific papers and citation data appear only as a small part rather than as the whole. The literature reports that the scientific models do better on scientific tasks for this reason (Singh et al., 2023), and the papers used here are scientific abstracts, so they ought to have the advantage. Whether they actually do is what the accomplished experiment tried to establish, and answering a question of that form does not require many models, only a strong representative of each kind.

The three candidates follow from that. all-MiniLM-L6-v2 represents the models built for text in general. Tt is the standard small model of the Sentence-BERT family (Reimers and Gurevych, 2019; Wang et al., 2020) and was downloaded more than 160 million times to the date of this study being written (Sentence-Transformers, 2021). It is general only in the sense of its intended use, however, and not in the sense of never having read science: its training mixture contains a substantial body of scientific papers, along with a portion of the very citation data that SPECTER was built on (Sentence-Transformers, 2021). This turns

out to matter for the result, and is returned to in Section 5.5. It is also the smallest model in the comparison, which makes it a challenging competitor when it comes to compute time. SPECTER2 represents the scientific models: it is the successor of SPECTER, the best known model of its kind (Cohan et al., 2020), begins from SciBERT, and is then trained on more than six million pairs of papers in which one cites the other, with its own documentation recommending it over the original for new work (Singh et al., 2023). Aspire is a second scientific model, included because it learns from a different signal, namely the sentences in which two papers are mentioned together, which state in words how the two are related (Mysore, Cohan, and Hope, 2022). If only one scientific model had been tested, a poor result could always be blamed on that one model. Testing two that learned in different ways means the outcome says something about scientific models in general, and not about one of them in particular.

The third question is practical. Every candidate had to be free to use and had to run on the available laptop graphics card, which has 6 GB of memory. This excludes the commercial embedding services, which charge for use, limit how much text may be sent, and can be replaced by their provider at any time, so that the same experiment run later might no longer give the same result. It also excludes models too large to fit on the card.

5.2 The Test

The test uses the paraphrase set described in Section 4.3, which contains 250 abstracts that were each rewritten as a single sentence in plain everyday English. The idea behind it is simple. A model is shown one of these sentences and asked to find the paper it was written from, among all 5,286 papers in the dataset. If the model understands what the sentence means, the correct paper should appear at or near the top of its list, so the position at which that paper appears is recorded and called its rank. A rank of one means the model found the right paper immediately, and lower ranks are therefore better.

In practice, all 5,286 abstracts are embedded once and the 250 rewritten sentences are embedded once, after which each sentence is compared against every abstract and the abstracts are sorted by how close they are to it. The 250 papers are split into five experiments of fifty, with no paper used twice, so that it becomes visible whether a model is consistently better or merely fortunate on one particular set. Each model is then described by three numbers: the average rank it achieves, the number of times it reaches rank one, and the time it needs to embed the dataset.

For a comparison to be meaningful, each model must be given the same job, otherwise a model might win simply because it was given an easier one. Every model therefore received exactly the same text: the abstract, on its own, with the title left out. SPECTER2's documentation suggests giving it the title together with the abstract, but doing so would have given one model more of the paper than the others, and any difference in the results could then no longer be attributed to the model itself. Every model was also timed on the

same graphics card, and the scripts stop with an error rather than moving the work to the processor if that card is unavailable, since a model timed on different hardware could not be compared with the others at all.

Each model was, however, allowed to compare texts in its own way. Every model measures how close two pieces of text are, but not all of them measure it in the same manner: MiniLM compares the direction of two vectors, whereas SPECTER2 and Aspire compare the distance between them, which is how each was trained. Each model was therefore judged by the measure its own authors intended, since a model asked to use a different one would be at a disadvantage that has nothing to do with its understanding of the text.

5.3 Hardening the Test

The first version of this test did not work, and the failure is described here because repairing it forms part of the method. In that version the paraphrases were written at the same length and in the same style as the originals, and all three models placed every one of the 250 papers at rank one. A test that everything passes is of no use, since telling the models apart is the only thing it has to do.

The primary issue was the accessibility of key scientific words. A search using the distinctive technical terms of an abstract still returned the correct paper at rank one, while a search using only plain everyday language did not. The models were therefore matching words rather than meaning, which is far easier than it appears: an unusual technical term occurs in only a handful of abstracts out of 5,286, so a paraphrase that keeps such terms can be traced back to its original without understanding anything about it. The instruction given to the paraphrase model was rewritten so that its wording could not be reused:

```
Rewrite the following scientific abstract as ONE single sentence using only
plain everyday English.  Imagine you are explaining this paper to a smart
14-year-old who has never read a scientific paper.
Strict rules:  do not reuse any technical terms, jargon, method names, model
names, algorithm names, dataset names, acronyms or proper nouns from the
original; replace any specialized vocabulary with plain descriptions; use
common everyday words throughout; write exactly ONE sentence; capture what
the paper does, why it matters, and what it found; return only the rewritten
sentence.
```

A small trial of 25 paraphrases confirmed that the models no longer scored perfectly and could be told apart. Only then was the full set of 250 generated, frozen, and reused unchanged, as described in Section 4.3.

5.4 Results

Table 5.2 reports the final results. MiniLM achieves the best average rank, 96.64 against 193.81 for SPECTER2 and 221.16 for Aspire, and it does so in each of the five experiments separately rather than only on average. It also finds the correct paper immediately more often than either specialist, 76 times out of 250 against 48 and 44, and it embeds the whole dataset in 7.1 seconds where the specialists need over twenty minutes, a difference of more than a hundred and fifty times. No statistical significance test was run on this comparison, because the differences are large, point in the same direction in every one of the five experiments, and are accompanied by a difference in running time that would decide the matter on its own.

Table 5.2: Results over the 250 paraphrases. Lower rank is better.

Model	Avg. rank	Rank-1 hits	Embed time
all-MiniLM-L6-v2	96.64	76/250 (30.4%)	7.1 s
SPECTER2	193.81	48/250 (19.2%)	1,236.2 s
aspire-sentence-embedder	221.16	44/250 (17.6%)	1,126.4 s

That a general-purpose model should beat two specialists is a surprising result, and its explanation lies in what the model was trained on. MiniLM is not purely general: of the 1.17 billion pairs of texts it was trained on, roughly 210 million, about eighteen percent, come from a corpus of scientific papers, together with a set of citation pairs from SPECTER itself (Sentence-Transformers, 2021). It has therefore read a great deal of science while also reading a great deal of ordinary English. The authors of SciRepEval report the same pattern from the other direction, finding that general-purpose models outperform SciBERT on their benchmark and attributing this to exactly this kind of scientific data appearing in general training sets (Singh et al., 2023). SPECTER2 and Aspire, in contrast, learned only from scientific citation signals (Singh et al., 2023; Mysore, Cohan, and Hope, 2022). The queries used in this test are written in plain English while the papers they must be matched to are written in technical language, which suggests that a model familiar with both kinds of text is better placed to connect them. The test differs in this respect from the task the tool actually performs, where abstracts are compared with other abstracts rather than with plain sentences, and this difference is taken up in Chapter 10.

5.5 The Decision

The model of choice is therefore all-MiniLM-L6-v2, and it meets both of the criteria set out at the beginning of this chapter. It is the more accurate of the three, achieving the best average rank overall and in each of the five experiments separately, so accuracy is not being given up in exchange for speed. It is also the faster model by a margin that matters in

practice rather than merely on paper. The tool embeds every collection of papers it is given, and the difference between seven seconds and twenty minutes is the difference between a tool an organiser can run again and again while trying out different room sizes, and one that has to be started and returned to later.

Each model was used exactly as its authors released it, including the amount of text it reads at once, which is 256 word pieces for MiniLM and 512 for the other two. MiniLM therefore reads slightly less of the longest abstracts than the other two models do, and it produced the better result nonetheless.

Chapter 6

Clustering

Chapter 5 finished with every paper turned into a vector, and Chapter 7 will begin with small groups of papers being packed into rooms. This chapter is the intermediate stage between the two: it takes the 5,286 papers and sorts them into small groups of similar papers, and these groups become the building blocks of the layout. What similar means was set forth in the previous chapter: two papers about the same topic receive vectors that lie close together, so grouping papers by topic is now a question of which vectors that are close to each other in value.

The reason for grouping at all is the size of the problem. Suppose the papers were placed one at a time. Each paper can go into any of the twelve rooms, so there are thousands of choices to make, and the choices depend on each other: every paper that takes a seat leaves one seat fewer for all the papers still waiting, and a room filled early with the wrong papers cannot be undone without moving everything placed after it. Finding the best layout would mean trying combinations of these choices, and the number of possible combinations is beyond what any computer could search through. Grouping first cuts the problem down to a manageable size: instead of asking where each of 5,286 papers should go, the next chapter only has to ask where each of a few hundred groups should go, and a question of that size can be answered well.

This type of approach is an established idea already. In route planning, a well known family of methods first groups the customers by location and then plans a route within each group, specifically because the full problem is too large to solve in one piece, and this approach produced excellent results and was extended by many later works (Fisher and Jaikumar, 1981). The same two-step idea has also been applied to conference scheduling itself (Bulhões, Correia, and Subramanian, 2022).

The grouping must fulfill three requirements. Every paper must end up in a group, because every paper needs a seat. Every paper must end up in exactly one group, because a paper can only stand in one room. And the method must work unchanged on any conference the tool is given, not only on this one.

6.1 The Method: Ward's Hierarchical Clustering

The method chosen is hierarchical clustering with Ward's rule (Ward, 1963). The idea is a loop that does one thing over and over. At the start, every paper stands alone, so there are 5,286 groups of one. At every step, the method looks at every possible pair of groups and asks the same question for each: if these two were merged, would the papers of the new group still sit close around their average, or would they be spread out? It then performs the one merge where the papers stay closest together, writes that merge down, and repeats. Ward's rule is the name of this test (Ward, 1963): it always tries the merge first in imagination, measures how spread out the combined group would be, and only carries out the merge that adds the least spread. After 5,285 such merges everything has been joined into one, and the written record of all the merges forms a tree (Murtagh and Contreras, 2012).

The number of groups follows one fixed rule, applied unchanged to any dataset the tool is given: the number of papers divided by ten, rounded to a whole number. For this conference that gives 5,286 / 10 = 529 groups. Why ten is the right divisor is explained in the next section.

Three properties of this method fit the task exactly. The first is that the tree is built once and can then be cut at any group count without building anything again. Table 6.1 shows what this costs in practice: building the tree for all 5,286 papers takes about two seconds, and cutting it takes six thousandths of a second, so trying a different number of groups is essentially free. The implementation saves the tree to disk and re-cuts it instantly. The second is that the method contains no randomness: the same papers produce the same tree and the same groups every single time it is run. This matters beyond convenience, because it makes every number in this dissertation reproducible, which is one of the goals set in Chapter 3. The third is that it adjusts to the data on its own: the group count is computed from the number of papers rather than fixed by hand, and since cutting is free, the very same tree serves whatever count a corpus requires.

Table 6.1: What the clustering costs, for all 5,286 papers.

Step	**Time**
Building the tree	1.99 s
Cutting it into 529 groups	0.006 s
Full script, everything included	3.57 s

Three common alternatives were considered and set aside, and Table 6.2 lists what each one is missing. This is not a claim that these methods are worse in general; each simply lacks one thing this task cannot give up. K-means is the most widely used clustering method (MacQueen, 1967; Jain, 2010). It must be told the number of groups before it starts, and it begins from starting positions chosen at random, so running it twice on the same papers can produce two different groupings. In contrast, Ward needs no number in advance and

contains no random step, so its result is the same on every run. HDBSCAN removes the need to choose a number of groups, as it finds them on its own (Campello, Moulavi, and Sander, 2013; McInnes, Healy, and Astels, 2017). It does this, however, by declaring papers that fit nowhere to be noise and leaving them out, and a poster session cannot do that: every paper must receive a seat. Topic models such as LDA take a different view of what a group is (Blei, Ng, and Jordan, 2003). Instead of placing each paper in one group, they describe every paper as belonging partly to several topics at once, for example half to one topic and half to another. That description may well be the more truthful one, but a paper can only stand in one room, so the assignment must be whole or nothing.

Table 6.2: Common alternatives and the required property each one lacks.

Method	**What this task needs that it does not give**
K-means	a fixed result: it starts from random positions
HDBSCAN	a group for every paper: it declares some papers noise
LDA	one group per paper: it gives each paper several topics

6.2 How Many Groups

The tree is cut into one group per ten papers, which for this conference gives 5,286 / 10, rounded to 529 groups. This is deliberately far more groups than there are rooms, and the reason lies in the next chapter. If the papers were cut into exactly twelve groups, one per room, the layout would already be decided before the optimization ever ran, and a single oversized group could exceed every room. Cutting into many small groups instead hands Chapter 7 a large set of small, coherent building blocks and the freedom to combine them well.

The choice of ten papers per group balances two failure directions. Toward fewer, larger groups, the blocks become rigid: each one commits many papers at once, the optimizer has fewer ways to fill the rooms tightly, and the largest blocks risk not fitting anywhere. Toward more, smaller groups, the clustering stops doing its job: in the extreme of one paper per group, the placement problem is back to its original 5,286 pieces and nothing was gained. Groups of around ten sit between the two: the clustering does the cheap and safe part of the work, joining papers that clearly belong together, and leaves the difficult judgement of what shares a room to the optimizer.

Because the count is computed from the number of papers rather than chosen by hand, the same rule carries over to any conference the tool is given. The implementation also computes silhouette scores for cuts between 5 and 50 groups as background; the decision was not based on them, for the reason given in the next section.

6.3 The Result

Table 6.3 summarizes the outcome of cutting the tree. The 529 groups average ten papers each, exactly as intended, the smallest holds 2 papers and the largest 44, and no paper is left alone in a group of one. Every paper sits in exactly one group, so the first two requirements are met by construction. Each group is then represented by its centre, the average of its papers' vectors, and these centres are the objects the next chapter works with: two groups whose centres lie close are two groups that would sit well in the same room.

Table 6.3: The 529 groups in numbers.

Papers	5,286
Groups	529
Smallest group	2 papers
Largest group	44 papers
Average group	10.0 papers
Groups of one paper	0
Mean cohesion	0.796
Cohesion range	0.692 – 0.925
Groups with cohesion ≥ 0.75	484 of 529
Silhouette (3,000-paper sample)	0.045

The quality of the groups is measured by their cohesion: for each group, the average closeness of its papers to the group's centre, where 1 would mean the papers all point the same way. The mean cohesion across the 529 groups is 0.796, the loosest group still reaches 0.692, and 484 of the 529 groups sit at 0.75 or higher, so the groups are genuinely about one thing each. This can also be checked by eye. Every group is labelled with the most typical words of its abstracts, and Table 6.4 shows a sample: the largest group is recognisably the literature on bandit algorithms, the next two are graph neural networks and reinforcement-learning-based reasoning, and even the loosest group of all still has a readable theme.

Table 6.4: Example groups: the three largest, the tightest, and the loosest.

Group	**Size**	**Cohesion**	**Most typical words**
495	44	0.805	regret, bandit, bandits, bound
188	29	0.771	graph, gnns, node, graphs
426	27	0.820	reasoning, rl, rlvr, models
161	2	0.912	carbon, ai, operational, footprint
475	12	0.692	social, users, collective, agent

One number in Table 6.3 reads poorly and is reported anyway: the silhouette score of 0.045. The silhouette rewards two things at once: papers being close within their group, and

the groups being far apart from each other (Rousseeuw, 1987). The groups here are not far apart, and that is by design: cutting 5,286 papers into 529 groups splits one broad topic into several small neighbouring groups, and neighbours sit close, so a measure that rewards distance between groups must score this low.

Chapter 7

Room Assignment

With the papers clustered, one thing is left: deciding which groups share which room. The ideal way to make such a decision is full optimization, meaning a method that searches the problem exactly and returns two things at once. It is the best possible layout, and a mathematical proof that no better layout exists. The proof is what makes this the gold standard. Without a proof, every layout leaves the same doubt: maybe more searching would have found a better one, and nobody can say how much better. With a proof, the doubt is gone: the search stops because nothing better exists. Exact methods for this class of problem exist (Grötschel and Wakabayashi, 1989), and modern solvers are regularly applied to it (Du et al., 2022), so trying the exact way first is the normal choice. This chapter starts with that attempt, then states the problem, reports what happened, and explains the road taken instead.

7.1 The Problem, Written Out

The room-assignment problem can be written in four steps.

1. **Set up the groups, rooms and similarities.**

   The notation of Chapter 3 carries over. There are $C = 529$ groups with sizes $s_1, \ldots, s_C$, and $R = 12$ rooms with seat capacities $c_1, \ldots, c_R$.

   The table W holds the similarity between every pair of group centres, computed from the embeddings of Chapter 5.

   A layout assigns every group i to exactly one room, written

   $$a_i \in \{1, \ldots, R\}.$$

   Every room must also hold at least m papers, with $m = 1$ here, so that no room stands empty.

2. **Give each layout a score.**

   The goal is to give every possible layout one score and make that score as large as possible. The score comes from the similarities between the groups that end up in the same room.

   The similarity between two groups i and j is stored as W_{ij}. The groups are represented by the centres of their paper embeddings, and these vectors can be compared by their direction using cosine similarity. This is the standard comparison used for Sentence-BERT embeddings, where texts with similar meaning are trained to have similar vector representations (Reimers and Gurevych, 2019). A higher value of W_{ij} therefore means that the two groups are more similar, while a lower value means that they are less similar.

   The optimisation then looks at where the groups are placed. If groups i and j are in the same room, their value W_{ij} is added to the score. If they are in different rooms, that pair adds nothing. This is repeated for every pair of groups in the layout.

   For example, if two groups that share a room have a similarity of 0.8, then 0.8 is added to the score. If another pair in the same room has a similarity of 0.3, another 0.3 is added. The room has therefore added 1.1 from those two pairs. The same process is repeated across all rooms.

   Formally,

$$f(a) \;=\; \sum_{\substack{1 \le i < j \le C \\ a_i = a_j}} W_{ij}. \tag{7.1}$$

   Here, $f(a)$ is the score of the complete layout a. The symbols i and j refer to two groups, while C is the total number of groups. The condition $i < j$ makes sure that each pair is counted only once. The condition $a_i = a_j$ means that the similarity W_{ij} is included only when the two groups are assigned to the same room.

   The optimiser changes the room assignments and keeps recalculating this total. A layout receives a higher score when it places more similar groups together. Maximising $f(a)$ therefore means finding the legal room assignment with the largest total similarity between the groups that share rooms.

3. **Represent the room-assignment problem mathematically.**

   The room-assignment problem can also be written as a set of mathematical decisions and rules. The basic decision is simple: for every created cluster, the solver has to choose one room.

   This is written using the binary variable y_{ir}. A binary variable can only be 0 or 1. Here,

$$y_{ir} = 1$$

means that group i is placed in room r, while $y_{ir} = 0$ means that it is not. For example, $y_{5,2} = 1$ means that group 5 is placed in room 2.

The solver considers these choices for every group and every room. It does not decide where one group should go on its own. It considers the room choices together and looks for an assignment that respects the room capacities and gives the largest overall similarity.

A second binary variable is used to show whether two groups end up together. The variable $z_{ijr} = 1$ when groups i and j are both in room r, and $z_{ijr} = 0$ otherwise. This matters because the similarity W_{ij} should only count when the two groups actually share a room. In the objective, $W_{ij} z_{ijr}$ therefore adds their similarity when they are together and adds zero when they are not.

The complete model is

$$\begin{aligned}
\max \quad & \sum_{i<j} \sum_{r} W_{ij}\, z_{ijr} \\
\text{s.t.} \quad & \sum_{r} y_{ir} = 1 \qquad \forall i, \\
& \sum_{i} s_i y_{ir} \le c_r \qquad \forall r, \\
& \sum_{i} s_i y_{ir} \ge m \qquad \forall r, \\
& z_{ijr} \le y_{ir}, \qquad z_{ijr} \le y_{jr}, \\
& z_{ijr} \ge y_{ir} + y_{jr} - 1, \\
& y_{ir},\, z_{ijr} \in \{0, 1\}.
\end{aligned}$$

The first line is the score the model tries to maximise. For every pair of groups that shares a room, their similarity W_{ij} is added. A layout therefore scores higher when it places more similar groups together.

The next line,

$$\sum_{r} y_{ir} = 1,$$

means that every group must be placed in exactly one room. If group 5 is placed in room 2, then $y_{5,2} = 1$ and its variables for the other rooms must be 0.

The capacity rule,

$$\sum_i s_i y_{ir} \leq c_r,$$

adds the sizes of all groups placed in room r and checks that they do not exceed the number of available seats. Here, s_i is the number of papers in group i and c_r is the capacity of room r.

The next rule,

$$\sum_i s_i y_{ir} \geq m,$$

makes sure that every room contains at least the required minimum number of papers. In this study $m = 1$, so this simply means that no room can be empty.

The next three rules work together to decide when two groups count as sharing a room. The first,

$$z_{ijr} \leq y_{ir},$$

means that z_{ijr} cannot be 1 unless group i is in room r. The second,

$$z_{ijr} \leq y_{jr},$$

does the same for group j. Together, these two rules mean that if either group is somewhere else, z_{ijr} must be 0 and their similarity does not count for that room.

The third rule,

$$z_{ijr} \geq y_{ir} + y_{jr} - 1,$$

covers the case where both groups are in the room. If group i and group j are both in room r, then $y_{ir} = 1$ and $y_{jr} = 1$. The right-hand side therefore becomes $1 + 1 - 1 = 1$, which forces $z_{ijr} = 1$. Their similarity W_{ij} is then included in the score.

Finally,

$$y_{ir},\ z_{ijr} \in \{0, 1\},$$

means that both variables can only be 0 or 1. In simple terms, every decision is a yes-or-no decision: a group is either in a room or it is not, and two groups either share a room or they do not.

The model therefore looks for an assignment of all groups that keeps every room within its capacity, leaves no room empty, and gives the largest total similarity between groups that share rooms. The exact solver works with this mathematical formulation directly. The heuristics use the same objective and room rules, but search for good layouts using their own procedures instead.

Two facts about this model matter for everything that follows.

First, the problem is a known hard one. It belongs to the family of clique partitioning problems, which are NP-hard (Grötschel and Wakabayashi, 1989), meaning no method is known that solves large cases quickly.

Second, the case here is genuinely large. With 529 groups there are 139,656 pairs, and with twelve rooms the model contains over 1.6 million yes-or-no decisions.

The number of possible layouts is astronomically beyond what could ever be listed and checked.

7.2 The Exact Attempt

The exact model was implemented in CP-SAT, the constraint solver of Google's OR-Tools library (Perron and Didier, 2025), and run on the full problem. After four hours it had still not finished. The run was stopped without a proven result, and nothing suggested it would finish in any acceptable time. No exact experiment was conducted beyond this attempt. This outcome matches what the literature reports: exact solvers struggle on problems of this type as they grow (Du et al., 2022). The attempt settled the direction of the project, though not because anything is known about what organisers prefer. No survey was conducted on whether they would wait days for a proven best layout or trade some quality for speed, and this work does not claim to know. What is known is the nature of the job itself: the details of a conference change while it is being organised — rooms are added or lost, papers are withdrawn, capacities shift — and every such change makes the previous layout outdated. A tool useful under these conditions must be cheap to run again, so that the organiser can experiment, try a change, see its effect, and try another. A method that needs hours for a single answer cannot support this way of working, whatever the quality of that answer. What the task needs is a very good layout in minutes, produced as often as the details change. That is a different requirement, and it is met by a different family of methods.

7.3 The Turn to Heuristics

A heuristic is a method that searches for good solutions without promising the best one. It trades away the proof: instead of certainty, it offers a strong layout within whatever time it is given, and in practice this trade is the standard one for large hard problems (Blum and Roli, 2003).The trade fits this project for two reasons. First, the proof adds little here. The organiser's question is "give me a very good layout now", not "prove that nothing better exists". Conference papers also tend to share close topics, so many groups can stand in for one another and many different layouts reach almost the same quality; when very good layouts are this abundant, the step from very good to proven best changes very little. The tool of Chapter 8 must be cheap to run. A layout that costs a few minutes on ordinary hardware could realistically be offered as a cloud service without large compute expenses; one that costs hours of solver time could not. Third, these methods have a strong record on this problem class (Palubeckis, Ostreika, and Tomkevičius, 2014), so choosing them is established practice, not a gamble.

Heuristics come in distinctly different families, each built around a different core idea (Blum and Roli, 2003): some restart many times from fresh starting points, some take one long walk guided by memory, some escalate how hard they shake a stuck solution, some breed a population of solutions, and some hand parts of the problem back to an exact solver. Five methods were chosen for the comparison, one established representative of each family: GRASP for restarting (Feo and Resende, 1995), Tabu Search for the memory-guided walk (Glover and Laguna, 1997), Variable Neighborhood Search for escalating shakes (Mladenović and Hansen, 1997), a Memetic algorithm for the population approach (Moscato and Cotta, 2003), and a Matheuristic for the hybrid with the exact solver (Maniezzo, Stützle, and Voß, 2010). Covering the families rather than many variants of one idea means the conclusion of this chapter is about kinds of search, not about one lucky implementation, and each representative is the traditional, widely cited method of its family.

The comparison itself was built to be fair, and the rules are simple. Every method attacked the same problem: the same 529 groups, the same twelve rooms, the same score $f(a)$, the same rule that no room may be empty. Every method received the same time: 60 seconds of search per attempt, a length found to work well for experimentation after trying several values. Every method made twenty attempts, and the attempts were numbered with the same twenty seed values across all methods, so that attempt number three of one method can be paired against attempt number three of another. This pairing is what makes the statistical tests of Section 7.6 valid. One property of these methods makes the fixed time essential, unlike the exact solver, which stops when it has its proof, a heuristic has no natural finishing point and would search forever, so the stopping time is a decision made from outside.

7.4 The Five Methods

Each method is now described in turn: its idea in plain words, its settings, and its results over the twenty attempts. In every results table, the similarity is the average closeness of the papers that ended up sharing a room, and its random-assignment baseline on this problem is 0.304, so every method's layout should be read against that floor.

7.4.1 GRASP

GRASP builds a layout quickly, polishes it, throws it away, and starts again (Feo and Resende, 1995). The building step places groups one by one, and at each placement it chooses randomly among the few best-looking options rather than always the single best, so every restart produces a different starting layout. The polishing step then improves the layout by moving single groups between rooms and swapping pairs of groups until no such move helps any more. The best layout seen across all restarts is kept. Within the 60-second budget, the method completed 40.5 restarts per attempt on average.

Table 7.1: GRASP over 20 attempts of 60 seconds.

Best score	8,306.15
Average score	8,176.36
Spread (std)	76.6
Paper similarity	0.3431
Restarts per attempt	40.5

7.4.2 Tabu Search

Tabu Search improves one layout in a single long walk (Glover and Laguna, 1997). At every step it tries all small changes, moving one group to another room or swapping two groups, and takes the best of them. The catch is that a walk taking only improving steps soon reaches a dead end where no change helps. Tabu Search therefore also accepts worsening steps to walk out of dead ends, and to stop itself from simply undoing them again and circling, it forbids each move for the 12 steps after it is made. The best layout seen anywhere along the walk is kept.

7.4.3 Matheuristic

The Matheuristic mixes the two worlds: a heuristic that steers, and the exact solver doing the detailed work (Maniezzo, Stützle, and Voß, 2010). It rests on one fact from Section 7.3: CP-SAT fails on all 529 groups, yet solves twelve groups perfectly in a fraction of a second.

Table 7.2: Tabu Search over 20 attempts of 60 seconds.

Best score	8,279.70
Average score	7,595.77
Spread (std)	333.3
Paper similarity	0.3486
Memory length (tenure)	12

So the method feeds it twelve at a time. It freezes the whole layout, frees twelve groups, and lets CP-SAT place those twelve as well as they can possibly be placed. It locks that answer in, frees a different twelve, and solves again. 754 of these little perfect solves per attempt on average. The bet: many small perfect decisions add up to one strong layout.

Table 7.3: Matheuristic over 20 attempts of 60 seconds.

Best score	8,326.37
Average score	7,893.15
Spread (std)	302.6
Paper similarity	0.3374
Exact sub-solves per attempt	754

7.4.4 Variable Neighborhood Search

Variable Neighborhood Search is built on one idea: when stuck, shake harder (Mladenović and Hansen, 1997). A shake means moving a few randomly chosen groups to random rooms, deliberately and without checking whether it helps. The method polishes a layout with the same local moves as the other methods until nothing helps, then shakes it and polishes again. If the shake did not lead anywhere better, the next shake is larger, up to ten groups at once, dropping back to a single group whenever an improvement is found. Each attempt completed 121.3 of these shake-and-polish rounds on average.

Table 7.4: VNS over 20 attempts of 60 seconds.

Best score	8,286.03
Average score	7,610.40
Spread (std)	329.6
Paper similarity	0.3493
Shake rounds per attempt	121.3

7.4.5 Memetic Algorithm

The Memetic algorithm keeps a population of 20 layouts at once, each one a complete candidate seating plan (Moscato and Cotta, 2003). It improves them by breeding. Two layouts are picked as parents, and a child is built from them: for each group, the child takes the room that one parent or the other gave it, so the child is a new layout mixing both. Small random changes, called mutations, are then applied to the child, and, the memetic signature, the child is polished with local search before it may join the population. The weakest layouts are dropped, the next generation begins, and each attempt ran 119.5 generations on average. The best layout ever present in the population is kept.

Table 7.5: Memetic algorithm over 20 attempts of 60 seconds.

Best score	8,312.05
Average score	8,150.35
Spread (std)	110.2
Paper similarity	0.3432
Population size / generations	20 / 119.5

7.5 The Choice: GRASP

Table 7.6 compares the five methods under the same conditions. No proven optimum exists, so the gaps are measured against the best layout any method found, the Matheuristic's 8,326.37.

Table 7.6: All five methods over the same 20 attempts of 60 seconds. Gaps are to the best layout found, not to a proven optimum.

Method	**Best**	**Average**	**Spread**	**Gap best**	**Gap avg.**	**Similarity**
GRASP	8,306.15	8,176.36	76.6	0.24%	1.80%	0.3431
Tabu Search	8,279.70	7,595.77	333.3	0.56%	8.78%	0.3486
Matheuristic	8,326.37	7,893.15	302.6	0.00%	5.20%	0.3374
VNS	8,286.03	7,610.40	329.6	0.48%	8.60%	0.3493
Memetic	8,312.05	8,150.35	110.2	0.17%	2.11%	0.3432

All five methods reach roughly the same peak: their best attempts lie within about half a percent of each other. They differ in what they deliver on an ordinary attempt. GRASP and Memetic average 8,176 and 8,150 with small spreads, while Tabu Search and VNS average far lower with spreads about four times larger. One pattern runs the other way and is examined in Chapter 9: the two weakest methods on the objective produce the highest paper similarity.

Whether these differences are real was tested with the standard procedure for repeated algorithm comparisons (Demšar, 2006). Friedman's test confirms that the methods are not all equal ($\chi^2 = 49.855$, $p < 0.001$), and pairwise Wilcoxon tests with Holm correction (Holm, 1979; García and Herrera, 2008) show which pairs differ, in Table 7.7. Nine of the ten pairs differ significantly. The exception is GRASP against Memetic, at $p = 0.355$: the two leaders cannot be separated from each other, and both are better than the remaining three.

Table 7.7: Pairwise Wilcoxon tests, Holm-corrected. Values below 0.05 mean a real difference.

Pair		p **(Holm)**
GRASP	Memetic	0.355
GRASP	Matheuristic	0.003
GRASP	Tabu Search	< 0.001
GRASP	VNS	< 0.001
Memetic	Matheuristic	0.007
Memetic	Tabu Search	< 0.001
Memetic	VNS	< 0.001
Matheuristic	Tabu Search	< 0.001
Matheuristic	VNS	< 0.001
Tabu Search	VNS	0.043

The choice between them therefore rests on everything else, and all of it favours GRASP: the higher average of the two, 8,176 against 8,150, the smaller spread, 76.6 against 110.2, and a far simpler design, since GRASP builds, improves and restarts where Memetic needs a population, selection, breeding, mutation and local improvement. Where results cannot separate two methods, the simpler one is the safer choice. GRASP is therefore the method the final tool uses, and how it is configured there is described in Chapter 8.

Chapter 8

The Layout Tool

Every chapter up to this point settled one decision on its own. This chapter is where all of them come together into the program that was actually built and handed over.

What the tool does is simple to state. It takes a file of papers and a list of room sizes, and it gives back a seating plan. That is all the user has to provide, and none of the pipeline has to be understood to use it. A whole run is one command:

```
python main.py --papers my_papers.csv --rooms 700,600,550
```

If the program is started without those details, for example by pressing the run button of an editor, it asks for them instead of stopping with an error. The papers file can also be dragged straight into the terminal window, which types its path automatically. Windows and Mac paste such a path in two different ways, and both are cleaned up so that either one works.

What happens after that is one run of six steps, and those six steps are this whole dissertation carried out in order. First, the input is checked and the tool reports what it is about to do. Then every abstract is turned into a vector, using the model chosen in Chapter 5. After that the vectors are grouped: the tree of Chapter 6 is built and cut into small groups of similar papers. Next those groups are packed into the rooms with the method chosen in Chapter 7, which is run several times over so that the best result can be kept. The finished layout is then measured against a random assignment of the same shape. The last step is saving, where three files are written out: the seating plan itself, a summary of every room, and a short quality report. Nothing in between needs a human decision. Table 8.1 shows the whole sequence.

Table 8.1: One run of the tool from start to finish, and where each step was decided.

Step	**Goes in**	**Comes out**	**Decided in**
Check	papers file, room sizes	a usable input	this chapter
Embed	5,286 abstracts	5,286 vectors	Chapter 5
Cluster	5,286 vectors	529 groups	Chapter 6
Pack	529 groups, 12 rooms	a layout	Chapter 7
Measure	the layout	similarity and baseline	Chapter 3
Save	all of the above	three output files	this chapter

Two things are true of the tool on every run. The first is that it sets itself up. The two numbers that shape a run, how many groups to make and how long to search for, are worked out from the size of the file it was given rather than asked of the user, so the same program handles a small workshop and a large conference without being changed.

The second is that it repeats itself. This matters for two reasons. A tool that gives a different answer every time cannot really be checked, since an organiser who reruns it and gets something else has no way of telling whether the change came from something they did or from the tool itself. And every number reported in this dissertation has to be reproducible by anyone who runs the same files again. Table 8.2 goes through the run and separates the parts that come out the same every time from the one part that does not.

Table 8.2: What is fixed in a run of the tool, and what is not.

Part of the run	**Why**
Fixed, the same on every run	
Checking the input	rows are dropped by rule, not by chance
Working out the settings	both numbers come from the paper count
Embedding	the same abstract always gives the same vector
Clustering	Ward has no random step in it
Which layouts get built	the random choices are set in advance
Measuring the layout	plain arithmetic on the finished layout
The random baseline	the shuffles are seeded and then averaged
Not fixed	
How many get built	the search stops on the clock, so a faster computer gets through more of them

Only one part of the run is not fixed, and it is worth being clear about what that does and does not mean. The random choices inside the search are set in advance, so the layouts always come out in the same order: the first one built is always the same layout, the second one is always the same, and so on down the list. What is not fixed is how far down that list the tool gets, because it stops when the time is up rather than after a set number of tries. A

faster computer works through more layouts in the same 53 seconds than a slower one does. If the best layout of the run happens to be far down the list, a fast machine reaches it and a slow machine does not.

In practice this has not caused a difference. The same conference was run three times and all three runs gave back the same plan, with a paper similarity of 0.3395 against a baseline of 0.3040 each time. A stopping rule based on a fixed number of tries rather than on the clock would remove the last piece of variation altogether, at the cost of a run whose length can no longer be predicted, and this is picked up in Chapter 10.

The program is split across five files, listed in Table 8.3. Only `main.py` knows anything about files, folders and users. The other four are handed data and give data back, and they never open a file themselves. The reason for splitting it this way is that each step can then be replaced on its own: a different embedding model, or a different optimisation method, means changing one file, and nothing else in the tool needs to know about it.

Table 8.3: The five files of the tool and what each one does.

File	What it does
`main.py`	reads the input, checks it, runs the four steps, writes the results
`embeddings_miniLM.py`	turns every abstract into a vector
`hierarchical_clustering.py`	builds the tree and cuts it into groups
`GRASP.py`	packs the groups into the rooms
`layout_evaluation.py`	measures the finished layout

8.1 Input

The tool is meant for conference organisers rather than programmers. Implementation was completed with the purpose of making it as seemless as possible. Before any of the heavy process starts it checks wether the given input is correct for the task. Anything it finds wrong is explained in plain words, not left as a crash for the user to decode.

The first check is the papers csv file. It has to exist, and it has to have a column of abstracts. Table 8.4 lists the four columns the tool looks for, and the abstract is the only one it cannot manage without. A file holding nothing but a column of abstracts runs perfectly well; the other three only make the finished plan easier to read.

Table 8.4: The columns the tool looks for in the papers file.

Column	**Needed?**	**What it is for**
Abstract	yes	the text everything is computed from
Title	no	names the papers in the seating plan
Category	no	labels the rooms in the summary
ID	no	numbers the papers; row numbers are used otherwise

Finding the abstract column is made flexible, since paper files rarely name it the same way twice. It is called `Abstract` by default, and the name is matched while ignoring capitals and any spaces around it, so `abstract` and `ABSTRACT` are both picked up without the user having to say anything. If the user's csv abstract column is named something else entirely, the user can inform the program of the new naming convention. Adding `--abstract-col` followed by the right naming convention tells the progrma what to look for, so for example, a file whose abstracts sit in a column called `Summary` is run like this:

```
python main.py --papers my_papers.csv --rooms 700,600,550 --abstract-col "Summary"
```

The tool then looks for Summary instead of Abstract, and the program works with the same instructed way. The same process of adding new naming convention can also be done to the other columns as follows:

```
python main.py --papers my_papers.csv --rooms 700,600,550
   --abstract-col "Summary" --category-col "Field" --id-col "PaperNo"
```

Three things can go wrong with the abstract column, and the tool answers each in proportion to how bad it is. If no matching column is found, it cannot guess which one was meant, so it prints every column name the file does have and stops; seeing the real names is usually enough to correct the command. If the column is there but some of its cells are empty, those papers are dropped, since a blank abstract cannot be turned into a vector, and the tool prints how many it dropped rather than removing them quietly. If every cell is empty, nothing is left to work with and the run stops. The rule behind all three is the same: the tool stops only when it truly cannot continue, and whenever it can carry on it says exactly what it did.

The next checks are about the rooms. Room sizes have to be positive whole numbers, so a typo like 0 or a negative number is caught here. Then come the two checks that matter most, both about whether the papers and the rooms actually fit together. The rooms have to have at least as many seats as there are papers, since every paper needs somewhere to go, and if there are not enough the tool prints how many papers and how many seats there are and stops. The number of papers also has to be at least the number of rooms, since every room has to end up with at least one paper. These two catch opposite problems: the first stops a run with too many papers for the space, the second a run with too few papers to

fill the rooms, which is what would happen if eight papers were handed twelve rooms. Only once all of this passes does the tool proceed with it's functions.

Two more columns are sought for, but only to make the output readable. A column named `Title` is used to name the papers in the seating plan. A column of categories is looked for under the names conferences usually give it, such as *primary area*, *category*, *track* or *topic*, and it can also be named directly with a flag if it is called something unusual. The reason it is worth collecting is that it gives the organiser a quick way of judging the layout by eye. The room summary then shows the three most common categories in every room, so a glance is enough to see whether each room came out as one recognisable subject or as a mixture. These labels are never read by the tool itself, which means that the result wether good or bad, is a result produced by the tool simply based on abstract data.

8.2 Hierarchical Clustering

Two numbers shape a run, and the tool works both of them out from the file it was given. Nothing is asked of the user.

The first number is how many groups to make. The rule comes from Chapter 6: one group per ten papers. The tool adds two safety lines around it. It never makes fewer groups than there are rooms, since every room needs at least one group. And it never makes more groups than there are papers.

The second number is how long each try may search. The rule is one second per hundred papers, so more papers means more time. Two cut-offs sit around it. A very small file still gets at least 10 seconds, because anything shorter is too little search to produce a decent layout. A very large file gets at most 300 seconds per try, because anything longer would leave the user waiting on a run for half an hour or more. In between, the rule runs as it is: this conference has 5,286 papers, so every try gets 53 seconds. One thing to note is why the time is set differently here than in Chapter 7. The experimentation needed fairness. Five methods were being compared, so every method had to get exactly the same time, and it was fixed by hand at 60 seconds. The tool needs something else. It has to work on any conference it is given, so the time has to come from the input itself. The rule is one second per hundred papers, which puts this conference at 53 seconds, close to the 60 that worked well in the tests. Any other size gets its time in the same proportion, meaning twice the papers, twice the time. Table 8.5 shows what both rules come to on the two datasets the tool has been run on.

Table 8.5: The rules the tool works out by itself, and what they come to on each dataset.

Rule	**NeurIPS (5,286 papers)**	**arXiv sample (10,000 papers)**
groups = papers / 10	529 groups	1,000 groups
time = papers / 100 seconds	53 s per try	100 s per try
tries, best one kept	5	5
minimum papers per room	1	1

Chapter 6 already showed how the papers are grouped, and the tool changes nothing about that. The tree is built the same way and cut by the same rule of one group per ten papers. So the job of this step inside the tool is different. Grouping well was settled there. Here the step prepares everything the packing will touch.

The packing that follows never sees a paper. It only sees groups, and each group is shrunk down to two things, a size and a direction. The size says how many seats the group needs, and every room is checked against it so nothing overflows. The direction is one single vector for the whole group, made by averaging the vectors of its papers, and it stands for what the group is about. This shrinking is the reason of why our program spends less computational resources in the optimization step. The packing handles a few hundred of these instead of thousands of papers, and that is why it can finish in seconds.

Before the next pipeline step starts, the clustering phase also prepares the answers for the heuristics part. It will keep asking one question, how alike are these two groups. So the answer is worked out once for every pair of groups, 139,656 pairs here, and written into a table. Two small rules apply to it. Anything below zero is stored as zero, and a group gets zero with itself. Only real togetherness should earn points.

One more thing is remembered from the cut. Every paper's group number is kept, because the packing answers in groups while the final plan must answer in papers. When the papers are split into groups, the tool writes down each paper's group number, one line per paper. This list exists because of a gap. GRASP only says where a group goes. The final file must say where a paper goes. The list closes the gap. GRASP puts group 17 in Room 3, the list shows every paper with the number 17, and each of those papers is written into the file with Room 3 beside it. And because the list points at the paper's own row in the original table, its title and category are picked up from there and written into the same line. Done for every group, the result is the seating plan, one row per paper with its title, its room, and its field.

A small repair rounds the step off. A group larger than the largest room would make the whole task impossible, since it fits nowhere. If the cut ever produces one, the same tree is cut a little finer and checked again, until nothing oversized remains. Cutting a finished tree again is nearly free. On the two datasets of this work this repair never had to run.

8.3 Heuristics

The packing is done by GRASP, which was chosen in Chapter 7. This is how it runs inside the tool.

GRASP works by making a complete layout, then throwing it away and making another one. Each of these attempts is called a try. A try starts from nothing, places every group into a room, improves what it has made, and ends with one finished layout that could be handed to an organiser as it stands. The tool then keeps whichever try came out best. The reason for working this way rather than carefully building one layout is that a single attempt can easily settle into a poor arrangement, and starting fresh costs so little that it is worth doing many times over.

One try is made of two parts: building a layout, then improving it. The building part places whole groups into rooms, one group at a time, and it works through them from the biggest group down to the smallest. Big groups are the hardest to fit anywhere, so they get their pick while the rooms are still empty. It runs in two stages.

Stage one fills every room once. It goes through the rooms in turn and, for each one, puts in the biggest group that still fits. This is what keeps any room from ending up empty.

Stage two places everything that is left. For each group, the tool looks at every room that still has space and scores it: the score is how similar this group is to the groups already sitting in that room. A room already holding related work scores high, a room holding unrelated work scores low. The group then goes into one of the three highest-scoring rooms, picked at random rather than always the top one.

That random pick has a specific purpose. Without it, every try would follow the same choices and build exactly the same layout, so running five tries would give five identical answers and there would be nothing to choose between. Picking among the top three instead means each try takes a slightly different path and produces a different layout. Occasionally a try runs itself into a dead end, where a group is left over and no room has space for it. When that happens the half-built layout is thrown away and a new one is started.

The improving part takes the finished layout and makes small changes to it for as long as they help. There are two kinds of change: moving one group into a different room, and swapping two groups between their rooms. A change is kept only if it raises the score, and only if it leaves every room legal, meaning inside its capacity and not below its minimum. When neither kind of change helps any more, the layout is done.

Building and improving then start over inside the same try, again and again, until the time for that try is up. Each of these repeats is a restart, and the best layout any of them produced is what the try returns. Chapter 7 measured about forty restarts in a minute of search, so a try is not one layout but the best of a few dozen.

The tool runs five of these tries and keeps the best of the five. Five is enough because a user needs one good layout, not a fair comparison of methods, and a few long tries serve that better than many short ones. Running five also guards against one unlucky try, since

GRASP is random and any single try can come out lower than the rest. Each try starts from its own fixed number, so running the tool twice on the same file gives back the same five layouts.

One thing to note for reading the results in Chapter 9. GRASP is handed a function that measures paper similarity and it reports that number for every try, but it never optimises on it. What GRASP maximises is the group-level score from Chapter 7, and the paper similarity is only carried along for the report.

8.4 What the Tool Gives Back

Every run makes its own numbered folder, `Production 1`, `Production 2` and so on, and the tool counts upwards until it finds a number that is not taken yet. What this does is make sure no run can ever overwrite an earlier one, so a user who tries three different sets of room sizes still has all three results afterwards. Three files are written into the folder.

The first is `layout.csv`, which is the actual deliverable: one row per paper, giving its number, its title, the room it goes to, and its category if the file had one. The second is `room_summary.csv`, one row per room, giving its capacity, how many papers it got, how full it is, how many groups it holds, the average similarity of the papers inside it, and the three most common categories in that room. The third is `quality_report.txt`, a short summary of the whole run, which for the NeurIPS conference looks like this:

```
source:  neurips_accepted_submissions.csv
papers:  5286   rooms:  12   seats:  5286
clusters:  529 (~10 papers each)
method:  grasp, 5 tries x 53s, best kept

average within-room paper similarity:  0.3395
random-assignment baseline:      0.3040
lift over random:             +0.0355
```

The purpose of the report is mostly on its last three lines. A similarity number alone says nothing, because nobody knows what counts as high. So the tool also shuffles the papers into the same rooms at random, measures that, and prints both numbers with the gap between them. The gap shows the actual success and quality of our method. In one of the datasets, the whole tool run itself took five tries of 53 seconds, a little over four minutes of searching, plus a few seconds for embedding and clustering.

That result can also be held against the benchmark of Chapter 7, where GRASP was measured over twenty tries of 60 seconds and averaged a paper similarity of 0.3431. The tool reaches 0.3395 while spending about a fifth of that search time, so the settings it uses cost a small amount of quality and buy a lot of speed. The rule behind them is a starting point that was calibrated rather than proven: it was set so that a conference of this size gets a

budget close to the one that worked in the benchmark, and what it gives to conferences of very different sizes has not been measured. This is picked up again in Chapter 10.

8.5 The Demonstration

The tool was run on two datasets. The first one is the conference this whole work was built on. The second one is the demonstration set from Section 4.4, ten thousand papers taken from nine different fields of arXiv. Nothing in this dissertation was decided based on this second set. Both files were handed over exactly as they came, with no cleaning done to them. What this section does is show what came out, read straight from the tool's own output files. What these results mean is left for Chapter 9.

Table 8.6 starts by putting the two runs next to each other, using the headline numbers from each run's quality report.

Table 8.6: The two runs of the tool.

	NeurIPS	**arXiv sample**
Papers	5,286	10,000
Rooms	12	12
Seats	5,286	10,800
Groups made	529	1,000
Search per try	53 s	100 s
Similarity	0.3395	0.1735
Random baseline	0.3040	0.0658
Lift over random	+0.0355	+0.1077

Each run also writes a file called room summary, and the two tables below print those files as they are, with the original column names and the original values. What each column holds is simple. capacity is the seats of the room. fill_pct says how full it came out. n_clusters says how many groups it holds. avg_paper_similarity is the average closeness of the paper pairs inside it. top_categories gives the three subject labels that appear most among its papers, together with their counts. It stays in the files. Another thing to note is that both input files carried a subject label for every paper, the NeurIPS primary areas and the arXiv categories, and the tool found both columns on its own.

Table 8.7 is the file from the NeurIPS run. The input was neurips_accepted_submissions.csv with 5,286 papers, twelve rooms holding 5,286 seats, 529 groups, and GRASP with five tries of 53 seconds. Since the seats add up to exactly the number of papers, there was no spare room anywhere, and every room came out completely full.

Table 8.7: room_summary.csv as produced for the NeurIPS run.

room	capacity	papers	fill_pct	n_clusters	avg_paper_similarity	top_categories
Room 1	700	700	100.0	135	0.2655	deep_learning (162) applications (120) theory (117)
Room 2	600	600	100.0	68	0.3663	applications (235) deep_learning (182) machine_learning_for_sciences (68)
Room 3	550	550	100.0	59	0.3503	deep_learning (186) social_and_economic_aspects_of_machine_learning (77) applications (71)
Room 4	500	500	100.0	64	0.3113	theory (146) optimization (75) deep_learning (62)
Room 5	450	450	100.0	34	0.4286	applications (199) deep_learning (166) general_machine_learning (23)
Room 6	425	425	100.0	39	0.3036	deep_learning (114) theory (59) probabilistic_methods (55)
Room 7	400	400	100.0	33	0.3664	deep_learning (190) machine_learning_for_sciences (50) social_and_economic_aspects_of_machine_learning (45)
Room 8	380	380	100.0	24	0.4296	deep_learning (232) applications (63) social_and_economic_aspects_of_machine_learning (26)
Room 9	350	350	100.0	18	0.3540	applications (145) deep_learning (114) social_and_economic_aspects_of_machine_learning (33)
Room 10	320	320	100.0	15	0.3469	deep_learning (115) social_and_economic_aspects_of_machine_learning (61) theory (33)
Room 11	300	300	100.0	18	0.3379	deep_learning (112) probabilistic_methods (40) applications (39)
Room 12	311	311	100.0	22	0.3318	reinforcement_learning (73) deep_learning (50) optimization (43)

Table 8.8 is the file from the demonstration run. The input was papers_dataset_random_10K.csv with 10,000 papers, twelve rooms holding 10,800 seats, 1,000 groups, and GRASP with five tries of 100 seconds. This time the rooms hold 800 more seats than there are papers. Nine rooms still came out completely full, Room 12 came out 91 percent full, and Room 9 ended up holding a single group of four papers.

Table 8.8: room_summary.csv as produced for the demonstration run.

room	capacity	papers	fill_pct	n_clusters	avg_paper_similarity	top_categories
						stat.CO (162)
Room 1	1300	1300	100.0	175	0.1586	stat.ME (152)
						econ.EM (147)
						eess.IV (256)
Room 2	1200	1200	100.0	100	0.1754	eess.AS (218)
						stat.ML (99)
						q-fin.PR (123)
Room 3	1100	1100	100.0	101	0.2413	q-fin.ST (102)
						q-fin.RM (102)
						econ.TH (224)
Room 4	1000	1000	100.0	78	0.1757	econ.GN (182)
						econ.EM (110)
						nlin.SI (191)
Room 5	950	950	100.0	121	0.1625	nlin.CD (82)
						nlin.PS (70)
						q-bio.MN (108)
Room 6	900	900	100.0	87	0.2006	q-bio.PE (83)
						q-bio.CB (82)
						math.GT (45)
Room 7	850	849	99.9	68	0.2010	math.AG (42)
						math.AC (42)
						nlin.PS (134)
Room 8	800	800	100.0	80	0.1719	nlin.AO (89)
						nlin.CD (75)
Room 9	750	4	0.5	1	0.3801	econ.GN (4)
						eess.SY (193)
Room 10	700	700	100.0	75	0.1299	eess.SP (102)
						cs.OS (30)
						eess.SP (47)
Room 11	650	650	100.0	82	0.0801	astro-ph.EP (28)
						q-bio.TO (27)
						eess.AS (66)
Room 12	600	547	91.2	32	0.0664	nlin.CD (47)
						stat.OT (23)

One thing matters when reading the last column. The pipeline never read any of these labels. It only read the abstracts. So when a whole room comes out as one field, the abstracts alone put it there. In the demonstration run this happened again and again, finance in Room 3, economics in Room 4, biology in Room 6, mathematics in Room 7. In the NeurIPS run the rooms mix areas instead, and deep learning shows up in almost every room. Why the two runs look this different, why the spare seats gathered in Room 9 and Room 12, and how the numbers of the two runs compare, is looked at in Chapter 9.

Chapter 9

Results, Discussion and Analysis

This work delivered two seating plans, one for each dataset. The first places the 5,286 papers of NeurIPS 2025 into twelve rooms whose seats add up to exactly 5,286. The second places 10,000 arXiv papers from nine fields into twelve halls with 10,800 seats. Both plans are legal. Every paper has a seat, no room is over its capacity, and no room is empty. Section 3.1 made five promises about what the pipeline delivers, and this section checks the plans against all five. All five promises are kept.

9.1 The Final Result

First, coherence. The NeurIPS plan scores a similarity of 0.3395 where a random assignment scores 0.3040, and the demonstration plan scores 0.1735 where random scores 0.0658. Both plans beat chance, and Section 9.6 shows that the second gap is the more impressive one. Second, feasibility. This promise is settled before any search starts, since the input checks of Chapter 8 reject every job that is impossible, and the packing keeps every rule while it works. A plan that breaks a seat count never comes into existence. Third, speed. Both plans took minutes on a laptop, since the tool searches with heuristics, while the one attempt to solve the problem exactly ran four hours and produced no answer. Fourth, generality. The second dataset arrived with different labels, a different size and a different balance of topics, and the tool adjusted every setting on its own and delivered. Fifth, reproducibility. The NeurIPS job ran three times and produced the same plan with the same numbers all three times.

Table 9.1: The five promises of Section 3.1 checked against the delivered plans.

Promise	**Evidence from the plans**	**Discussed in**
Coherent	0.3395 vs random 0.3040 on NeurIPS, 0.1735 vs 0.0658 on the demonstration	Sections 9.2, 9.6
Feasible	impossible inputs rejected, rules kept during search, both plans legal	Section 9.3
Fast	minutes for 5,286 and for 10,000 papers, four hours with no answer for the exact attempt	Section 9.4
General	second dataset handled with zero changes to the tool	Section 9.6
Reproducible	three identical plans from three identical runs	Chapter 10

Two of these measured results carry a condition, and stating the condition is part of reporting the result. The coherence numbers only mean something next to their random floors, since 0.3395 by itself is just a number, and the floor is what turns it into a distance. The repeatability was proven on one machine, since three runs on that computer gave one identical plan, while a faster computer finishes more layouts in the same time and sits outside this proof. Neither condition shrinks a result. Each one says where the result is established. The rest of the chapter now goes through everything in order. The rooms from the inside, the seat counts, the method choice and the time it holds for, the two quality scores, the real worth of the numbers, and at the end the results that went against expectation.

9.2 Similarity Analysis for Each Room

The similarity number for each room is stated but that doesn't tell what the rooms look like inside, so this section reads them individually. Two patterns appear across the tables of Chapter 8, and both come from the data rather than from the pipeline misbehaving.

Pattern one is that big rooms score lower than small rooms, and this is expected rather than a problem. A big room holds several subjects at once. That does not defeat what the layout is for. A visitor in that room still finds each subject standing in its own block, one area ending where the next begins, which is exactly the experience the pipeline promises. The number is lower only because of how an average works. A room's similarity averages every pair of papers inside it, and pairs between different subjects enter that average too. A small room holds one subject, so almost all its pairs are close, and the number comes out high. A big room holds several, so many pairs cross from one subject to another, and the number comes out lower. The reality in the tables matches this exactly. The tightest rooms of the NeurIPS plan are the small ones, Room 8 at 0.430 with 24 groups and Room 5 at 0.429 with 34. The loosest is the biggest, Room 1 at 0.266 with 135 groups, and its own

labels show deep learning, applications and theory sharing the hall in their own blocks. So the low number on Room 1 measures how much ground one average has to cover, not how well the papers were placed.

Pattern two is that deep learning appears in the top labels of every NeurIPS room, all twelve of them, and this is expected rather than a problem. One label covering a third of a conference has to appear everywhere. That does not defeat what the layout is for. The label is only a name, and underneath the name the papers were still sorted correctly, the deep learning papers about graphs sitting with the graph work, the ones about proofs sitting with the theory work. A visitor moving through those rooms meets coherent subjects, whatever the label column says. The reason the label spreads is simple arithmetic. It covers 1,685 papers, the biggest room holds 700 seats, so no assignment of any kind keeps it in one place. The reality in the tables confirms the sorting worked. On the second dataset plan, where every field is small enough to fit inside a room, the same tool delivers clean one-field rooms, finance in Room 3, economics in Room 4, biology in Room 6, mathematics in Room 7, on labels the pipeline never read. So one dataset shows the sorting when the labels are too big to stay together, and the other shows it when they are not, and both come from the same tool doing the same thing.

Two rooms of the second dataset plan follow a different logic, one nearly empty and one loose at 91 percent. They belong to the next section, since they come from how the spare seats were handled.

9.3 How the Heuristics Handled Exact Fill

Groups are frozen. Once the clustering makes them, the packer never splits one, never merges two, and never moves a single paper between them. The re-cutting of Chapter 8 runs before the packing and only when a group is too big for the biggest room, which never happened on either dataset. This section shows what packing whole frozen groups did under two very different seat situations, and it went well both times.

The NeurIPS run had zero spare seats, 5,286 papers for 5,286 chairs. Every room has to reach its exact count, and the packer only has whole groups of sizes 2 to 44 to do it with. At the start this is easy, since the rooms are empty and everything fits. At the end it is hard, since the open gaps are small. A gap of thirty seats takes a group of thirty or less and nothing else. So the last placements go by fit, not by topic, because closing the gaps is what makes the plan exist. The result is that all twelve rooms hit their exact counts. These forced placements likely land mostly in the biggest room, since that is where space runs out last, and they might add to the mix in Room 1 from the previous section. How much they add is unmeasured. Measuring it needs the same conference run again with spare seats, and that run does not exist. One way to remove the pressure entirely is cutting the groups finer, since smaller groups close small gaps easily, and Chapter 10 proposes exploring that.

The second dataset run had 800 more seats than papers. So unlike NeurIPS, where every

seat had to be used, here some seats were always going to stay empty. The only question was where the empty ones end up, and the tables answer with two separate facts.

Fact one is where the empty seats went. Nearly all into a single room. Room 9 got only 4 papers for its 750 seats, and Room 12 took the small remainder, ending at 91 percent. Because two rooms soaked up all the emptiness, the other ten stayed completely full.

Fact two is where the badly matching groups went. The two worst similarity scores of the whole plan sit in that same corner, 0.066 in Room 12 and 0.080 in Room 11. This suggests the groups with no good home anywhere also drifted to the tail. So both kinds of leftovers, the seats nobody needed and the groups nothing wanted, ended up in the same few rooms, and that is exactly why the other ten look clean.

This concentration is not necessarily the style an organiser wants, since a nearly empty room can look odd on the day. Chapter 10 discusses this and proposes a setting that lets the organiser decide how the empty seats spread.

9.4 How Well the Method Choice Held Up

GRASP was chosen in a 60 second race against four other methods in Chapter 7. Two properties make it fit a clock particularly well, and the delivered tool then confirmed the choice in practice.

The first property is that GRASP finishes whole layouts over and over. It builds a complete plan, improves it, stores it, and starts again, about 40 finished plans per benchmark minute. Stopping it at any moment still leaves a stack of complete answers to pick from. Tabu Search and VNS instead push one plan through a long chain of changes, and stopping them mid-chain catches whatever state the chain is in. The second property is steadiness. Across twenty timed attempts GRASP's scores vary by 76.6 while Tabu Search's vary by 333.3, more than four times as much. The Matheuristic shows a third behaviour, reaching the single best score of the whole benchmark, 8,326.37, while averaging only 7,893.15, strong when its exact steps finish in time and weak when they run long. A tool that promises an answer in minutes wants many finished plans and small swings, and that is GRASP's exact profile, together with the best average of the field at 8,176.36.

The tool then confirmed the choice. At 53 seconds per try and five tries, it reached a paper similarity of 0.3395 against the 0.3431 the benchmark's GRASP averaged, which is 99 percent of the quality at a fifth of the search time, and it delivered the identical plan on three consecutive runs. The reading is that GRASP near this time scale sits where extra search changes little, which is the right place for a fixed-budget tool to operate. One thing to note is the scope of the win. The race was run at 60 seconds, so GRASP is the proven winner at time scales like this one, which is the scale the tool works at. Other race lengths were not run, and instead of guessing about them, Chapter 10 proposes handing the time setting to the user.

The method is settled. What the method was told to chase is the next subject, since the score it climbs and the score a visitor feels are two different numbers.

9.5 Did the Tool Produce a Good Layout for Conferences?

This section answers the question in the title. The answer is yes, on both datasets, and the harder dataset gives the stronger yes. The section lays out how that answer is reached, step by step.

Step one is deciding what good means, and meeting the two measures. Papers joined together in a room is the mechanical outcome, but the real judgement is whether a visitor at the conference enjoys it and finds everything they care about in one place. That experience is not observable before the conference happens, so it is judged through the two strongest measures this work has. The objective, the group level score the search climbed while it packed, and the within room similarity, which averages how close the papers standing in each room are to each other.

Step two is why success is not judged on the objective value alone. A high objective is good, and there the certainty ends. Nothing says what high actually means, since there is no known maximum to hold the value against. The search is a heuristic, not a full optimization, so a better layout with a higher value might exist without anyone knowing, and Chapter 7 showed that computing the true maximum is out of reach, four hours without an answer. The value only says the heuristics performed well but not how well it performed.

Step three is why the similarity is the better measurement, and what it still needs. The similarity works on the papers themselves, from outside, independent of how the pipeline grouped anything, so it measures exactly the thing a visitor stands next to. But its value alone has the same reading problem in a different form, since nothing about 0.3395 says whether that is high or low. To place it, this work uses the random shuffle. The shuffle takes the same papers and the same room sizes, assigns the papers at random with no method at all, and measures the result the same way. What the shuffle scores is the floor, the part any layout gets for free, and a layout is judged by how far it stands above its floor.

Step four is the two layouts read against their random scores. Each dataset has its own random score, and the two are far apart for a clear reason. NeurIPS papers all come from one field, so even a random shuffle puts related papers together, and its random score is high, 0.3040. The second dataset mixes nine fields, so a random shuffle puts economists next to physicists, and its random score is low, 0.0658. The two random scores differ by a factor of almost five before any method does anything. Against these, NeurIPS finishes at 0.3395, which is 0.0355 above its random score, a climb of about twelve percent. The second dataset finishes at 0.1735, which is 0.1077 above, a climb of about 164 percent, more than two and a half times what chance scores. So the raw numbers point one way and the climbs point

the other. The layout with the smaller number did the larger work. On NeurIPS the papers were already similar, so random chance alone reached a high score, and the pipeline added a visible margin on top. On the second dataset random chance reached almost nothing, and the pipeline built nearly the whole result itself. The harder the data made the task, the more of the result belonged to the method. This is the strongest evidence in this dissertation that the pipeline works on data it has never seen, and it was earned on the dataset that played no part in any design decision.

All together, the answer to the question is yes. The NeurIPS layout scored twelve percent above its random score. The second dataset layout scored 164 percent above its random score. Both layouts beat chance, the second by far more, and the second dataset is the one that played no part in building the pipeline. The tool produced good layouts on both datasets.

9.6 Expectations vs Actual Outcomes

Three things looked certain when this work started. All three turned out wrong. Each one taught something.

The first was that models built for science win on science. They lost. SPECTER2 trained on six million citation pairs. Aspire matches sentences between papers. A small general model, all-MiniLM-L6-v2, beat both in all five tests of Chapter 5. Its average rank was 96.64. Theirs were 193.81 and 221.16. The reason is simple. MiniLM trained on everyday language rather than only scientific text, and it turns out that in this case this works best. The small model is also over 170 times faster. It embeds a whole conference in 7 seconds. The specialists took near twenty minutes.

The second was that complex methods beat simple ones. The numbers say otherwise. The most complex method got the single best score, 8,326.37. But on average it got 7,893.15. Plain GRASP got 8,176.36 on average, which is better. The other complex method got 8,150.35, and a test showed no real difference between it and GRASP, p equal to 0.355. GRASP was chosen for its structural simplicity and its better scores on some metrics.

The third was that exact optimization is worth it when possible. It is not possible considering the fact that if one conference organizer wants to adjust one small thing, they have to wait again for the endless hours of the full optimizer. The exact solver ran four hours on the full problem and proved nothing. That is NP-hard in practice.

These three outcomes also answer where this work fits. Two camps handle conference layouts today. The first computes the perfect plan and proves it, but the four hour run of Chapter 7 shows no organiser waits that long. The second, what large conferences actually do, builds a plan quickly with embeddings and uses it with no check at all. This work takes the speed of the second and the honesty of the first. It builds plans in minutes and puts a number on every plan by checking it against random. Chapter 2 argued every ingredient existed and nobody had connected them. That pipeline now exists, tested on the conference

it was built around and one it had never seen. Both plans kept every rule, with all papers seated and no room over its limit. Both were scored, each one against its own random shuffle. And the code behind all of it is public, so anyone reruns the whole pipeline and gets the same numbers. The final chapter covers what this work settled, what it left open, and what to build next.

Chapter 10

Conclusion

This dissertation asked whether a program can take any conference's papers and any set of room sizes and return a seating plan in which every room holds papers about similar things. The program built to answer it is a four-stage pipeline. It reads only the abstracts, turns each abstract into a vector, sorts the vectors into small groups of related papers, and packs whole groups into the rooms.

Every stage choice was defended against its alternatives. Three embedding models competed on the paraphrase test, and the small general model all-MiniLM-L6-v2 beat both scientific specialists on accuracy and on speed. Ward's hierarchical clustering was chosen since it places every paper in exactly one group, re-cuts to any group count at almost no cost, and returns the same groups on every run. The exact solver was tried first, ran four hours on the full problem without an answer, and the packing therefore went to heuristics. Five heuristic families raced under one fair protocol, the same groups, the same time, the same seeds, and GRASP was chosen for the best average, the smallest spread, and the simplest design.

The finished tool was run on two datasets. On the 5,286 papers of NeurIPS 2025 in twelve rooms it reached a within-room similarity of 0.3395 against a random score of 0.3040, about twelve percent above it. On the demonstration set of 10,000 arXiv papers from nine fields it reached 0.1735 against a random score of 0.0658, about 164 percent above it, earned on data that played no part in any design decision. Both plans kept every rule, with every paper seated, no room over its capacity, and no room empty. Both took minutes on a laptop, and on the reproducibility trials, the tool returned the same plan on all of its runs. The research question is answered. The pipeline produces topically coherent, physically feasible layouts at conference scale, and the harder the data makes the task, the larger the share of the result that belongs to the method.

10.1 Limitations

Six limitations bound what these results can claim. They follow the order of the pipeline, from what the tool reads to how long it searches.

The first limitation is partial abstract information. The pipeline reads nothing but the abstract, and MiniLM reads the first 256 word pieces of it, while the abstracts in the dataset run up to 516 words. For the longest abstracts the similarity is therefore computed from the opening part alone, and whatever an author says only later in the abstract, or only in the full paper, cannot influence where that paper stands.

The second limitation is that similarity is the model's judgement. Every quality number in this work is a closeness of vectors in MiniLM's space, and no person has checked whether a room the model calls coherent also feels coherent to a visitor walking through it. The model itself was also selected on a different task than the one the tool performs. The test of Chapter 5 matched plain sentences to abstracts, while the tool compares abstracts with abstracts, so the winner of the first task was trusted to be the winner of the second without a direct test.

The third limitation is that the cost of exact fill is unmeasured. On the NeurIPS run every seat had to be used, so the last placements went by fit rather than by topic. How much similarity those placements gave up is unknown, since measuring it needs the same conference run again with spare seats, and that run does not exist.

The fourth limitation is that spare seats gather where they will. On the demonstration run the 800 unneeded seats collected almost entirely in two rooms, one nearly empty and one at 91 percent. The tool gives the organiser no say in this, and a hall holding four papers may not be the layout an organiser wants on the day.

The fifth limitation is that reproducibility holds per machine. The search stops on the clock, so a faster computer finishes more restarts in the same seconds and can reach a layout a slower one never builds. Three runs on one machine returned one identical plan, and that is the exact scope of the guarantee.

The sixth limitation is that the time settings are calibrated at one scale. The benchmark evidence covers searches of sixty seconds, and the tool's rule of one second per hundred papers was set so that a conference of this size lands near that scale. What the rule gives a far smaller or far larger conference has not been checked.

10.2 Future Work

The future work answers all six limitations, in the order the work will happen, and ends with one extension.

For the second limitation, the study and the tool will be given to real conference organisers. A survey will collect their feedback and record whether the rooms the model calls coherent also feel coherent at their sessions. The survey will then be analysed to find which areas of the pipeline need improving or optimising.

For the first and the third limitation, the answers are held back on purpose. Both layouts stood above their random scores, so nothing yet says the reading or the grouping needs repair. Two signals would reopen them. The first is a substantial number of conferences coming back barely above their random scores or below them. The second is the survey analysis pointing at the reading or the grouping. If either case will be true, a study will be put in place to decide the measures needed.

For the fourth limitation, every room will get a minimum fill, a percentage the organiser chooses, so spare seats spread instead of gathering in one place. The model of Section 7.2 has already spoken about this, the minimum occupancy that today stands at one paper.

For the fifth and the sixth limitation we are planning the same solution. The tool will run as a cloud service, something Section 7.4 already mentioned, with every job running on the same rented hardware. A cloud service that will be computationally more capable then the hardware this study was tested on, which will guarantee the user identical and better quality of layouts. The user will have the option to also choose how much time should the tool consume to produce a layout. Better time means better layout but the smallest choice of time bracket will be chosen as such to give the user a well defined layout regardless, so even the cheapest choice returns a layout of tested quality.

Apart from future work on all the pre-mentioned limitations, this study is planning to be flexible with more details of an actual conference. A conference is not only rooms but also time. Large venues already spread their posters over several sessions across several days, with each paper standing in one session. The extension will let a conference hand the tool time periods next to room sizes, allowing a conference to optimize even with the time aspect in mind.

References

Beltagy, Iz, Kyle Lo, and Arman Cohan (2019). "SciBERT: A Pretrained Language Model for Scientific Text". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3615–3620. DOI: 10.18653/v1/D19-1371.

Blei, David M., Andrew Y. Ng, and Michael I. Jordan (2003). "Latent Dirichlet Allocation". In: *Journal of Machine Learning Research* 3, pp. 993–1022.

Blum, Christian and Andrea Roli (2003). "Metaheuristics in Combinatorial Optimization: Overview and Conceptual Comparison". In: *ACM Computing Surveys* 35.3, pp. 268–308. DOI: 10.1145/937503.937505.

Bulhões, Teobaldo, Rubens Correia, and Anand Subramanian (2022). "Conference scheduling: A clustering-based approach". In: *European Journal of Operational Research* 297.1, pp. 15–26. DOI: 10.1016/j.ejor.2021.04.042.

Burke, Michael and Deon Sabatta (2015). "Topic models for conference session assignment: organising PRASA 2014". In: *Proceedings of the 2015 Pattern Recognition Association of South Africa and Robotics and Mechatronics International Conference (PRASA-RobMech)*.

Campello, Ricardo J. G. B., Davoud Moulavi, and Jörg Sander (2013). "Density-Based Clustering Based on Hierarchical Density Estimates". In: *Advances in Knowledge Discovery and Data Mining (PAKDD)*, pp. 160–172. DOI: 10.1007/978-3-642-37456-2_14.

Cohan, Arman et al. (2020). "SPECTER: Document-level Representation Learning using Citation-informed Transformers". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*.

CVPR (2024). *CVPR 2024 Accepted Papers*. URL: https://cvpr.thecvf.com/Conferences/2024/AcceptedPapers (visited on 07/16/2026).

Demšar, Janez (2006). "Statistical Comparisons of Classifiers over Multiple Data Sets". In: *Journal of Machine Learning Research* 7, pp. 1–30.

Du, Yu et al. (2022). "Solving clique partitioning problems: A comparison of models and commercial solvers". In: *International Journal of Information Technology & Decision Making* 21, pp. 59–81. DOI: 10.1142/S0219622021500504.

Eglese, R. W. and G. K. Rand (1987). "Conference Seminar Timetabling". In: *Journal of the Operational Research Society* 38.7, pp. 591–598.

Feo, Thomas A. and Mauricio G. C. Resende (1995). "Greedy Randomized Adaptive Search Procedures". In: *Journal of Global Optimization* 6.2, pp. 109–133. DOI: 10.1007/BF01096763.

Fisher, Marshall L. and Ramchandran Jaikumar (1981). "A Generalized Assignment Heuristic for Vehicle Routing". In: *Networks* 11.2, pp. 109–124. DOI: 10.1002/net.3230110205.

Gao, Tianyu, Xingcheng Yao, and Danqi Chen (2021). "SimCSE: Simple Contrastive Learning of Sentence Embeddings". In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 6894–6910. DOI: 10.18653/v1/2021.emnlp-main.552.

García, Salvador and Francisco Herrera (2008). "An Extension on "Statistical Comparisons of Classifiers over Multiple Data Sets" for all Pairwise Comparisons". In: *Journal of Machine Learning Research* 9, pp. 2677–2694.

Glover, Fred and Manuel Laguna (1997). *Tabu Search*. Springer. DOI: 10.1007/978-1-4615-6089-0.

Grötschel, Martin and Yoshiko Wakabayashi (1989). "A cutting plane algorithm for a clustering problem". In: *Mathematical Programming* 45, pp. 59–96. DOI: 10.1007/BF01589097.

— (1990). "Facets of the clique partitioning polytope". In: *Mathematical Programming* 47, pp. 367–387.

Holm, Sture (1979). "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6.2, pp. 65–70.

Jain, Anil K. (2010). "Data clustering: 50 years beyond K-means". In: *Pattern Recognition Letters* 31.8, pp. 651–666.

MacQueen, James (1967). "Some methods for classification and analysis of multivariate observations". In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*. Vol. 1, pp. 281–297.

Maniezzo, Vittorio, Thomas Stützle, and Stefan Voß, eds. (2010). *Matheuristics: Hybridizing Metaheuristics and Mathematical Programming*. Vol. 10. Annals of Information Systems. Springer. DOI: 10.1007/978-1-4419-1306-7.

McInnes, Leland, John Healy, and Steve Astels (2017). "hdbscan: Hierarchical density based clustering". In: *Journal of Open Source Software* 2.11, p. 205. DOI: 10.21105/joss.00205.

Mladenović, Nenad and Pierre Hansen (1997). "Variable neighborhood search". In: *Computers & Operations Research* 24.11, pp. 1097–1100.

Moscato, Pablo and Carlos Cotta (2003). "A Gentle Introduction to Memetic Algorithms". In: *Handbook of Metaheuristics*. Ed. by Fred Glover and Gary A. Kochenberger. Kluwer.

Murtagh, Fionn and Pedro Contreras (2012). "Algorithms for hierarchical clustering: An overview". In: *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 2.1, pp. 86–97. DOI: 10.1002/widm.53.

Mysore, Sheshera, Arman Cohan, and Tom Hope (2022). "Multi-Vector Models with Textual Guidance for Fine-Grained Scientific Document Similarity". In: *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.

Oosten, Maarten, Jeroen H. G. C. Rutten, and Frits C. R. Spieksma (2001). "The clique partitioning problem: Facets and patching facets". In: *Networks* 38.4, pp. 209–226.

Ostendorff, Malte et al. (2022). "Neighborhood Contrastive Learning for Scientific Document Representations with Citation Embeddings". In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 11670–11688. DOI: 10.18653/v1/2022.emnlp-main.802.

Palubeckis, Gintaras, Armantas Ostreika, and Arūnas Tomkevičius (2014). "An Iterated Tabu Search Approach for the Clique Partitioning Problem". In: *The Scientific World Journal* 2014, p. 353101.

Perron, Laurent and Frédéric Didier (2025). *CP-SAT*. Google. URL: https://developers.google.com/optimization/cp/cp_solver/ (visited on 07/21/2026).

Reimers, Nils and Iryna Gurevych (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992.

Riquelme, Fabián et al. (2022). "A Track-Based Conference Scheduling Problem". In: *Mathematics* 10.21, p. 3976.

Rousseeuw, Peter J. (1987). "Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis". In: *Journal of Computational and Applied Mathematics* 20, pp. 53–65. DOI: 10.1016/0377-0427(87)90125-7.

Sentence-Transformers (2021). *all-MiniLM-L6-v2 model card*. URL: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 (visited on 07/20/2026).

Singh, Amanpreet et al. (2023). "SciRepEval: A Multi-Format Benchmark for Scientific Document Representations". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.

Stidsen, Thomas, David Pisinger, and Daniele Vigo (2018). "Scheduling EURO-k conferences". In: *European Journal of Operational Research* 270.3, pp. 1138–1147.

Wang, Wenhui et al. (2020). "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers". In: *Advances in Neural Information Processing Systems*. Vol. 33.

Ward, Joe H. (1963). "Hierarchical Grouping to Optimize an Objective Function". In: *Journal of the American Statistical Association* 58.301, pp. 236–244. DOI: 10.1080/01621459.1963.10500845.